

รายงาน

การเปรียบเทียบการทดสอบโดยใช้ปัญญาประดิษฐ์กับอัลกอริทึมการสร้างกรณีทดสอบอัตโนมัติ: การประเมินประสิทธิภาพและความครอบคลุมของการทดสอบ
AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A
Benchmark and Test Coverage Evaluation

จัดทำโดย

673380286-0 นางสาวพิไลพร คำเวียง

673380297-5 นางสาวหทัยพัช วิสุทธีธรรม

อาจารย์ที่ปรึกษา ผศ. ดร.ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา รหัส CP353201 การประกันคุณภาพซอฟต์แวร์

สาขาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบประสิทธิภาพของเทคนิคการสร้างชุดทดสอบซอฟต์แวร์แบบอัตโนมัติระหว่างแนวทางที่ใช้อัลกอริทึมการค้นหาและแนวทางที่ใช้ปัญญาประดิษฐ์ (Artificial Intelligence: AI) โดยดำเนินการทดลองกับโครงการซอฟต์แวร์จากชุดข้อมูล Defects4J ซึ่งเป็นชุดข้อมูลที่รวบรวมข้อบกพร่องของซอฟต์แวร์ Java สำหรับการวิจัยด้านการทดสอบซอฟต์แวร์ การทดลองมุ่งเน้นการประเมินประสิทธิภาพของชุดทดสอบที่สร้างขึ้นในด้านความครอบคลุมของโค้ด ได้แก่ Line Coverage และ Condition Coverage รวมถึงความสามารถในการตรวจจับข้อบกพร่อง (Fault Detection) และความสามารถในการสร้างชุดทดสอบที่สามารถคอมไพล์และทำงานได้จริง

ผลการทดลองถูกนำมาวิเคราะห์และเปรียบเทียบระหว่างแนวทางการสร้างชุดทดสอบ เพื่อพิจารณาความแตกต่างด้านความครอบคลุม คุณภาพและความสามารถในการใช้งานของชุดทดสอบ ตลอดจนความสามารถในการตรวจจับข้อบกพร่อง ผลการศึกษาช่วยให้เห็นถึงจุดเด่น ข้อจำกัด และลักษณะการทำงานของแต่ละแนวทางในการสร้างชุดทดสอบแบบอัตโนมัติ ซึ่งสามารถนำไปใช้เป็นข้อมูลประกอบการประยุกต์ใช้เทคนิคการสร้างชุดทดสอบอัตโนมัติและปัญญาประดิษฐ์ในกระบวนการประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance: SQA) และการพัฒนาซอฟต์แวร์ต่อไป

คำสำคัญ: การประกันคุณภาพซอฟต์แวร์, การทดสอบซอฟต์แวร์, การสร้างชุดทดสอบอัตโนมัติ, ปัญญาประดิษฐ์, ความครอบคลุมของการทดสอบ, การตรวจจับข้อบกพร่อง, Defects4J

ABSTRACT

This project aims to study and compare the effectiveness of automated software test suite generation techniques between algorithm-based approaches and Artificial Intelligence (AI)-based approaches. The experiments are conducted using software projects from the Defects4J dataset, which contains real-world Java software defects and is widely used in software testing research. The evaluation focuses on several performance metrics, including Line Coverage, Condition Coverage, Fault Detection, and the ability to generate test suites that can successfully compile and execute.

The experimental results are analyzed and compared to examine differences in code coverage, the quality and executability of generated test suites, and their ability to detect software faults. The study provides insights into the strengths and limitations of each test generation approach and can serve as a basis for selecting and applying automated test generation and AI-based techniques to improve the efficiency of Software Quality Assurance (SQA) processes and software development.

Keywords: Software Quality Assurance, Software Testing, Automated Test Generation, Artificial Intelligence, Test Coverage, Fault Detection, Defects4J

กิตติกรรมประกาศ

โครงการนี้สำเร็จลุล่วงได้ด้วยความกรุณาและความช่วยเหลือจากบุคคลหลายฝ่าย ผู้จัดทำจึงขอขอบคุณไว้ ณ โอกาสนี้ ที่กรุณาให้คำแนะนำ แนวทางในการดำเนินงานตลอดจนข้อเสนอแนะที่เป็นประโยชน์ต่อการศึกษา การทำการทดลอง

ขอขอบพระคุณ ผศ. ดร.ชิตสุธา สุ่มเล็กที่ได้ให้ความรู้ คำชี้แนะและแนวทางในการดำเนินงานของโครงการนี้ ขอขอบคุณเพื่อน ๆ ที่ให้ความช่วยเหลือและให้คำปรึกษาไปจนถึงการให้กำลังใจตลอดระยะเวลาดำเนินโครงการ จนทำให้โครงการฉบับนี้สำเร็จลุล่วงไปได้ด้วยดี

ผู้จัดทำ
หทัยพัทธ วิสุทธธรรม
พีเลพร คำเวียง

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance) สาขาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น จัดทำขึ้นเพื่อศึกษาและเปรียบเทียบประสิทธิภาพของการสร้างชุดทดสอบซอฟต์แวร์แบบอัตโนมัติระหว่างแนวทางที่ใช้อัลกอริทึมกับแนวทางที่ใช้ปัญญาประดิษฐ์ โดยเลือกศึกษา 4 วิธี ได้แก่ Grey Wolf Optimizer (GWO) ที่ประยุกต์ใช้ร่วมกับ EvoSuite, Zest ซึ่งเป็นเทคนิค Coverage-Guided Fuzzing และโมเดลภาษาขนาดใหญ่ 2 โมเดล ได้แก่ Gemini-3.7-flash และ DeepSeek-v4-flash และทดลองกับข้อบกพร่องจริงจากชุดข้อมูล Defects4J จำนวน 17 โครงการ

เนื้อหาในรายงานประกอบด้วยความเป็นมาและวัตถุประสงค์ของโครงการ งานวิจัยและทฤษฎีที่เกี่ยวข้อง วิธีการดำเนินงานและเครื่องมือที่ใช้ ผลการทดลองด้าน Line Coverage, Branch Coverage และ Condition Coverage รวมถึงคุณลักษณะของชุดทดสอบ การวิเคราะห์สาเหตุความล้มเหลวในการสร้างชุดทดสอบด้วยปัญญาประดิษฐ์ ตลอดจนการอภิปรายผล สรุปผล ปัญหาอุปสรรค และข้อเสนอแนะสำหรับการศึกษาต่อไป

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้ที่สนใจด้านการทดสอบซอฟต์แวร์และการประยุกต์ใช้ปัญญาประดิษฐ์ในกระบวนการประกันคุณภาพซอฟต์แวร์ หากรายงานฉบับนี้มีข้อผิดพลาดประการใด ผู้จัดทำขออภัยไว้ ณ ที่นี้ และยินดีรับข้อเสนอแนะเพื่อนำไปปรับปรุงต่อไป

ผู้จัดทำ

หทัยพัชร วิสุทธธรรม

พิไลพร คำเวียง

สารบัญ

บทคัดย่อภาษาไทย.....	ก
บทคัดย่อภาษาอังกฤษ	ข
กิตติกรรมประกาศ.....	ค
คำนำ.....	ง
สารบัญ	จ
สารบัญตาราง.....	ช
สารบัญภาพ.....	ซ
บทที่ 1 บทนำ.....	1
1. ความเป็นมาและความสำคัญของปัญหา.....	1
2. วัตถุประสงค์ของโครงการ.....	1
3. เป้าหมายและขอบเขต.....	1
4. ประโยชน์ที่คาดว่าจะได้รับ.....	2
บทที่ 2 งานวิจัยและทฤษฎีที่เกี่ยวข้อง.....	3
1. งานวิจัยที่เกี่ยวข้อง.....	3
2. ทฤษฎีที่เกี่ยวข้อง.....	5
บทที่ 3 วิธีการดำเนินงาน	7
1. รูปแบบการทดลอง	7
2. เครื่องมือและสภาพแวดล้อม.....	8
3. ชุดข้อมูล Defects4J	9
4. ขั้นตอนการทดลอง	9
5. กระบวนการการทำงาน.....	10
6. ตัวชี้วัดที่ใช้ในการประเมินผล	11
บทที่ 4 ผลการทดลองและการวิเคราะห์.....	12
1. ภาพรวมการทดลอง	12

2.	ผลด้าน Line Coverage	13
3.	ผลด้าน Branch และ Condition Coverage	18
4.	การวิเคราะห์สาเหตุความล้มเหลวในการสร้างชุดทดสอบด้วยปัญญาประดิษฐ์	23
5.	ผลด้าน Test Suite Characteristics.....	23
6.	อภิปรายผล	29
บทที่ 5 สรุปผลการทดสอบ		32
1.	สรุปผลการทดสอบ	32
2.	ปัญหาอุปสรรค และแนวทางการแก้ไข	34
3.	ข้อเสนอแนะ	36

สารบัญตาราง

ตารางที่ 1 เครื่องมือและสภาพแวดล้อม.....	8
ตารางที่ 2 จำนวน Bug ที่สามารถทดสอบได้ด้วย GWO, Zest, Gemini และ DeepSeek จำแนกตามโครงการ Defects4J.....	12
ตารางที่ 3 ค่าเฉลี่ย Line Coverage จำแนกตามโครงการ Defects4J และวิธีการทดสอบ.....	13
ตารางที่ 4 ค่าเฉลี่ย Line Coverage ของ Common Bugs จำแนกตามโครงการ Defects4J และวิธีการทดสอบ.....	15
ตารางที่ 5 ค่าเฉลี่ย Branch Coverage ของ GWO และ Zest จำแนกตามโปรเจค Defects4J และจำนวน Common Bugs.....	18
ตารางที่ 6 ค่าเฉลี่ย Condition Coverage ของ Gemini และ DeepSeek จำแนกตามโปรเจค Defects4J และจำนวน Common Bugs	20
ตารางที่ 7 ค่าเฉลี่ยคุณลักษณะของชุดกรณีทดสอบที่สร้างโดย Grey Wolf Optimizer จำแนกตามโครงการ Defects4J	24
ตารางที่ 8 ค่าเฉลี่ยคุณลักษณะของชุดกรณีทดสอบที่สร้างโดย Zest จำแนกตามโครงการ Defects4J.....	25
ตารางที่ 9 คุณลักษณะการสร้างกรณีทดสอบด้วย Gemini จำแนกตามโครงการ Defects4J.....	26
ตารางที่ 10 คุณลักษณะการสร้างกรณีทดสอบด้วย DeepSeek จำแนกตามโครงการ Defects4J.....	28
ตารางที่ 11 ภาพรวมค่าเฉลี่ยรวมทุก Project.....	33

สารบัญภาพ

ภาพที่ 1 ค่าเฉลี่ย Line Coverage จำแนกตามโครงการ Defects4J และวิธีการทดสอบ.....	14
ภาพที่ 2 กราฟค่าเฉลี่ย Line Coverage ของ Common Bugs จำแนกตามโครงการ Defects4J และวิธีการทดสอบ.....	16
ภาพที่ 3 กราฟแสดงภาพรวม Line Coverage ของแต่ละวิธี.....	17
ภาพที่ 4 กราฟแสดงการเปรียบเทียบ Branch Coverage ของ GWO และ Zest จำแนกตามโปรเจค.....	19
ภาพที่ 5 กราฟแสดงการเปรียบเทียบภาพรวมของ Branch Coverage ระหว่าง GWO และ Zest.....	20
ภาพที่ 6 กราฟแสดงการเปรียบเทียบ Condition Coverage ระหว่าง Gemini และ DeepSeek.....	22
ภาพที่ 7 กราฟแสดงภาพรวม Condition Coverage ของ Gemini และ DeepSeek	22

บทที่ 1

บทนำ

1. ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบันซอฟต์แวร์มีบทบาทสำคัญต่อการดำเนินงานในหลายด้าน ทั้งระบบสารสนเทศ ระบบธุรกิจ และอื่น ๆ ส่งผลให้ความถูกต้องของซอฟต์แวร์เป็นปัจจัยสำคัญในการพัฒนาซอฟต์แวร์ การทดสอบซอฟต์แวร์จึงเป็นกระบวนการสำคัญในการประกันคุณภาพซอฟต์แวร์ เนื่องจากช่วยตรวจสอบความถูกต้องของการทำงานและค้นหาข้อบกพร่องที่อาจเกิดในระบบ อย่างไรก็ตามการสร้างกรณีทดสอบด้วยตนเองอาจต้องใช้เวลานาน โดยเฉพาะในระบบที่มีความซับซ้อน

ปัจจุบันมีการพัฒนาเครื่องมือสำหรับการสร้างกรณีทดสอบโดยอัตโนมัติหลายแนวทาง เช่น Search based Software Testing, Coverage-guided Fuzzing และการใช้แบบจำลองปัญญาประดิษฐ์เพื่อสร้างกรณีทดสอบ ซึ่งแต่ละแนวทางมีกลไกในการสร้างและค้นหากรณีทดสอบที่แตกต่างกัน

ดังนั้น โครงการนี้จึงทำการทดลองเพื่อศึกษาและเปรียบเทียบประสิทธิภาพของการสร้างชุดทดสอบซอฟต์แวร์แบบอัตโนมัติระหว่างแนวทางที่ใช้อัลกอริทึมและแนวทางที่ใช้ปัญญาประดิษฐ์

2. วัตถุประสงค์ของโครงการ

- 2.1 เพื่อศึกษาและทดลองใช้เครื่องมือสร้าง Test Case โดยอัตโนมัติด้วยแนวทางที่แตกต่างกัน
- 2.2 เพื่อเปรียบเทียบประสิทธิภาพของเครื่องมือในการสร้าง Test Case

3. เป้าหมายและขอบเขต

3.1 ใช้ชุดข้อมูลสำหรับการทดลอง

3.1.1 ใช้ชุดข้อมูล Defects4J ซึ่งเป็นชุดข้อมูลข้อบกพร่องของโครงการซอฟต์แวร์ภาษา Java สำหรับใช้เป็นมาตรฐานในการทดลองและประเมินประสิทธิภาพของเทคนิคการสร้างชุดทดสอบ โดยใช้โปรแกรมและข้อบกพร่องที่กำหนดไว้ในชุดข้อมูลเป็นกรณีศึกษา

3.2 การพัฒนาอัลกอริทึม Grey Wolf Optimizer ใน EvoSuite

3.2.1 พัฒนาและประยุกต์ใช้อัลกอริทึม Grey Wolf Optimizer (GWO) เข้ากับเครื่องมือ EvoSuite เพื่อใช้ในการค้นหาและสร้างชุดทดสอบโดยอัตโนมัติ โดยเปรียบเทียบประสิทธิภาพของแนวทางดังกล่าวกับแนวทางการสร้างชุดทดสอบอื่นภายใต้ชุดข้อมูลและเงื่อนไขการทดลองที่กำหนด

3.3 การใช้ Zest สำหรับการสร้างชุดทดสอบ

3.3.1 ใช้ Zest ซึ่งเป็นเครื่องมือสำหรับการสร้างข้อมูลทดสอบแบบอัตโนมัติด้วยแนวทาง Fuzzing เพื่อสร้างชุดทดสอบสำหรับโครงการที่อยู่ในชุดข้อมูล Defects4J และนำผลลัพธ์ที่ได้มาประเมินตามเกณฑ์ที่กำหนด

3.4 การใช้ปัญญาประดิษฐ์ในการสร้างชุดทดสอบ

3.4.1 ประยุกต์ใช้โมเดลปัญญาประดิษฐ์สำหรับการสร้างชุดทดสอบซอฟต์แวร์ ได้แก่ Gemini-3.7-flash และ DeepSeek-v4-flash โดยให้โมเดลสร้างชุดทดสอบสำหรับโปรแกรม Java ภายใต้ข้อกำหนดและรูปแบบการทดลองที่กำหนด จากนั้นนำชุดทดสอบที่สร้างขึ้นมาประเมินประสิทธิภาพเช่นเดียวกับแนวทางอื่น

4. ประโยชน์ที่คาดว่าจะได้รับ

- 4.1 ได้ข้อมูลสำหรับเปรียบเทียบประสิทธิภาพของเทคนิคการสร้างชุดทดสอบแบบอัตโนมัติ ได้แก่ Grey wolf optimizer, Zest และ AI
- 4.2 ได้แนวทางในการประยุกต์ใช้ AI และอัลกอริทึมสำหรับการสร้างชุดทดสอบซอฟต์แวร์
- 4.3 สามารถนำผลการศึกษาไปใช้เป็นแนวทางในการพัฒนากระบวนการ Software Quality Assurance ต่อไป

บทที่ 2

งานวิจัยและทฤษฎีที่เกี่ยวข้อง

1. งานวิจัยที่เกี่ยวข้อง

- 1.1 Padhye, R., Lemieux, C., Sen, K., Papadakis, M., และ Le Traon, Y. (2019) ได้ศึกษาและพัฒนา Zest ซึ่งเป็นแนวทางการทำ Semantic Fuzzing สำหรับการสร้างข้อมูลทดสอบแบบอัตโนมัติ โดยใช้ข้อมูลจากโครงสร้างและความหมายของโปรแกรมร่วมกับแนวทาง Coverage-Guided Fuzzing เพื่อค้นหาข้อมูลทดสอบที่สามารถเพิ่มความครอบคลุมของโปรแกรมและช่วยค้นหาข้อบกพร่องได้ งานวิจัยนี้แนะนำแนวทางที่ช่วยเพิ่มประสิทธิภาพของการสร้างข้อมูลทดสอบ และแสดงให้เห็นถึงความสามารถของ Semantic Fuzzing ในการนำไปประยุกต์ใช้กับการทดสอบซอฟต์แวร์
- 1.2 Efficient Test Case Reduction Using Grey Wolf Optimization Techniques. (2025) ได้ศึกษาแนวทางการลดจำนวนกรณีทดสอบ (Test Case Reduction) โดยประยุกต์ใช้เทคนิค Grey Wolf Optimization (GWO) เพื่อเลือกกรณีทดสอบที่มีความสำคัญและลดความซ้ำซ้อนของชุดทดสอบ งานวิจัยมุ่งเน้นการรักษาศักยภาพในการตรวจสอบซอฟต์แวร์ในขณะที่ลดขนาดของชุดทดสอบ ซึ่งช่วยลดเวลาและทรัพยากรที่ใช้ในการดำเนินการทดสอบ ผลการศึกษาสะท้อนให้เห็นว่าอัลกอริทึม GWO สามารถนำมาประยุกต์ใช้กับปัญหาการเพิ่มประสิทธิภาพที่เกี่ยวข้องกับกรณีทดสอบได้
- 1.3 Kumari, S., Jindal, S., และ Sharma, A. (2025) ได้ศึกษาแนวทางการเพิ่มประสิทธิภาพกรณีทดสอบ (Test Case Optimization) โดยประยุกต์ใช้อัลกอริทึม Grey Wolf Optimization Algorithm เพื่อค้นหาชุดกรณีทดสอบที่เหมาะสมสำหรับกระบวนการทดสอบซอฟต์แวร์ งานวิจัยมุ่งเน้นการประยุกต์ใช้หลักการ Optimization เพื่อปรับปรุงประสิทธิภาพของชุดทดสอบและลดต้นทุนของกระบวนการทดสอบ ผลการศึกษาชี้ให้เห็นถึงความเป็นไปได้ในการนำ GWO มาใช้เป็นเทคนิคสำหรับแก้ปัญหาการเพิ่มประสิทธิภาพในงานทดสอบซอฟต์แวร์ และสามารถใช้เป็นแนวทางสำหรับการศึกษาการประยุกต์ใช้อัลกอริทึม GWO กับการสร้างหรือปรับปรุงชุดทดสอบในงานวิจัยต่อไป
- 1.4 Kang, S., Yoon, J., และ Yoo, S. (2023) ได้ศึกษาการประยุกต์ใช้ Large Language Models (LLMs) สำหรับการทดสอบซอฟต์แวร์ โดยมุ่งเน้นความสามารถของ LLM ในการสร้างข้อมูลและกรณีทดสอบสำหรับการทำซ้ำข้อบกพร่อง (Bug Reproduction) งานวิจัยศึกษาการใช้ตัวอย่างจำนวนไม่มาก (Few-Shot) เพื่อให้โมเดลสามารถสร้างกรณีทดสอบที่เกี่ยวข้องกับข้อบกพร่องของซอฟต์แวร์ ผลการศึกษาชี้ให้เห็นถึงศักยภาพของ LLM ในการสนับสนุนกระบวนการทดสอบและการสร้างกรณีทดสอบสำหรับข้อบกพร่องของซอฟต์แวร์

- 1.5 Plein, L., Ouédraogo, W. C., Klein, J., และ Bissyandé, T. F. (2024)** ได้ศึกษาความเป็นไปได้ในการใช้ Large Language Models เพื่อสร้างกรณีทดสอบจากรายงานข้อบกพร่อง (Bug Reports) โดยนำข้อมูลจากรายงานข้อบกพร่องมาใช้เป็นข้อมูลประกอบการสร้าง Test Case งานวิจัยมุ่งศึกษาว่า LLM สามารถนำข้อมูลเพื่ออธิบายลักษณะของข้อบกพร่องมาใช้ในการสร้างกรณีทดสอบที่สามารถนำไปตรวจสอบข้อบกพร่องดังกล่าวได้หรือไม่ ผลการศึกษาช่วยแสดงให้เห็นถึงแนวทางการนำ LLM มาใช้ร่วมกับข้อมูลจากระบบการพัฒนาซอฟต์แวร์เพื่อสนับสนุนการทดสอบแบบอัตโนมัติ
- 1.6 Schäfer, M., Nadi, S., Eghbali, A., และ Tip, F. (2023)** ได้ทำการประเมินเชิงประจักษ์เกี่ยวกับการใช้ Large Language Models สำหรับการสร้าง Unit Test แบบอัตโนมัติ โดยศึกษาความสามารถของโมเดลในการสร้างชุดทดสอบสำหรับโปรแกรมซอฟต์แวร์ และประเมินผลในด้านต่าง ๆ เช่น ความถูกต้องของชุดทดสอบ ความสามารถในการคอมไพล์ และความครอบคลุมของโค้ด งานวิจัยดังกล่าวแสดงให้เห็นถึงศักยภาพและข้อจำกัดของ LLM ในการสร้าง Unit Test และเป็นพื้นฐานสำคัญสำหรับการศึกษการประยุกต์ใช้ LLM ในงาน Automated Test Generation
- 1.7 Ouedraogo, W. C., Kabore, A. K., Li, Y., Tian, H., Koyuncu, A., Klein, J., Lo, D., และ Bissyandé, T. F. (2026)** ได้ศึกษาผลกระทบของ Prompt Engineering ต่อการสร้าง Unit Test แบบอัตโนมัติด้วย Large Language Models โดยทำการศึกษาในขนาดใหญ่เพื่อเปรียบเทียบรูปแบบและเทคนิคการออกแบบ Prompt ที่แตกต่างกันในการสร้างชุดทดสอบ งานวิจัยมุ่งศึกษาว่าการกำหนด Prompt ที่เหมาะสมสามารถส่งผลกระทบต่อคุณภาพของชุดทดสอบที่สร้างโดย LLM ได้อย่างไร ผลการศึกษาช่วยสนับสนุนแนวคิดว่ากรออกแบบ Prompt เป็นปัจจัยสำคัญที่ควรพิจารณาในการนำ LLM มาใช้สำหรับ Automated Unit Test Generation
- 1.8 Wang, G., Xu, Q., Briand, L. C., Guo, Z., และ Liu, K. (2026)** ได้ศึกษาแนวทาง Call-Chain-Aware LLM-Based Test Generation สำหรับโครงการภาษา Java โดยนำข้อมูลเกี่ยวกับลำดับการเรียกใช้งานเมธอด (Call Chain) มาใช้เป็นบริบทประกอบการสร้างกรณีทดสอบด้วย Large Language Models แนวทางดังกล่าวมีเป้าหมายเพื่อช่วยให้โมเดลเข้าใจความสัมพันธ์และการทำงานร่วมกันของส่วนต่าง ๆ ภายในโปรแกรมได้ดีขึ้น และนำข้อมูลดังกล่าวมาใช้ในการสร้าง Test Case สำหรับโปรแกรม Java งานวิจัยนี้แสดงให้เห็นถึงแนวทางการเพิ่มข้อมูลบริบทของโปรแกรมเพื่อช่วยปรับปรุงกระบวนการสร้างชุดทดสอบด้วย LLM

2. ทฤษฎีที่เกี่ยวข้อง

2.1 การทดสอบซอฟต์แวร์

การทดสอบซอฟต์แวร์เป็นกระบวนการตรวจสอบและประเมินซอฟต์แวร์เพื่อค้นหาข้อผิดพลาดและตรวจสอบว่าซอฟต์แวร์สามารถทำงานได้ตามข้อกำหนดที่กำหนดไว้ การทดสอบสามารถทำได้หลายระดับ เช่น Unit Testing, Integration Testing และ System Testing โดยโครงการนี้มุ่งเน้นการสร้างและประเมินชุดทดสอบในระดับ Unit Test สำหรับโปรแกรมภาษา Java

2.2 การสร้างชุดทดสอบแบบอัตโนมัติ

การสร้างชุดทดสอบแบบอัตโนมัติเป็นกระบวนการใช้เครื่องมือหรืออัลกอริทึมในการสร้าง Test Case โดยลดการสร้างกรณีทดสอบด้วยมนุษย์ เทคนิคดังกล่าวสามารถช่วยลดเวลาและทรัพยากรในการทดสอบ และสามารถสร้าง Test Case จำนวนมากเพื่อสำรวจพฤติกรรมของโปรแกรม โดยโครงการนี้ศึกษาการสร้างชุดทดสอบจากหลายแนวทาง ได้แก่ EvoSuite ที่ประยุกต์ใช้ GWO, Zest และ AI

2.3 Defects4J

Defects4J เป็นชุดข้อมูล Benchmark สำหรับการวิจัยด้านการทดสอบและการวิเคราะห์โปรแกรมภาษา Java โดยรวบรวมโครงการซอฟต์แวร์ที่มีข้อบกพร่องจริงและจัดเตรียมเวอร์ชันก่อนและหลังการแก้ไขข้อบกพร่อง ชุดข้อมูลดังกล่าวช่วยให้นักวิจัยสามารถทดลองและเปรียบเทียบประสิทธิภาพของเทคนิคการสร้างชุดทดสอบภายใต้กรณีทดลองเดียวกันได้ โครงการนี้ใช้ Defects4J เป็นชุดข้อมูลหลักในการทดลอง

2.4 EvoSuite

EvoSuite เป็นเครื่องมือสำหรับการสร้าง Unit Test แบบอัตโนมัติสำหรับโปรแกรมภาษา Java โดยใช้แนวคิด Search-Based Software Testing และ Evolutionary Algorithm เพื่อค้นหาชุดทดสอบที่เหมาะสมตาม Fitness Function ที่กำหนด เช่น ความครอบคลุมของโค้ด ในโครงการนี้มีการนำ Grey Wolf Optimizer มาประยุกต์ใช้ร่วมกับ EvoSuite เพื่อใช้เป็นแนวทางในการค้นหาชุดทดสอบ

2.5 Grey Wolf Optimizer (GWO)

Grey Wolf Optimizer เป็นอัลกอริทึมการหาค่าเหมาะที่สุดที่ได้รับแรงบันดาลใจจากพฤติกรรมการล่าเหยื่อและโครงสร้างทางสังคมของหมาป่าสีเทา โดยแบ่งลำดับชั้นของฝูงหมาป่าเป็น Alpha, Beta, Delta และ Omega และใช้ตำแหน่งของหมาป่าที่มีผลลัพธ์ดีที่สุดเป็นแนวทางในการปรับตำแหน่งของสมาชิกในประชากร

ในโครงการนี้ GWO ถูกนำมาประยุกต์ใช้กับ EvoSuite เพื่อค้นหาชุดทดสอบที่มีประสิทธิภาพ โดยใช้ค่าความเหมาะสม (Fitness) และข้อมูลจากการทำงานของชุดทดสอบเป็นข้อมูลสำหรับกระบวนการค้นหา

2.6 Zest และ JQF

Zest เป็นแนวทางการสร้างข้อมูลทดสอบแบบ Coverage-Guided Fuzzing และ Semantic Fuzzing โดยใช้ข้อมูลจากการทำงานของโปรแกรม เช่น Code Coverage เพื่อช่วยเลือกและสร้างข้อมูลทดสอบใหม่ ส่วน JQF (Java QuickCheck Framework) เป็นเฟรมเวิร์กสำหรับ Property-Based Testing ในภาษา Java ที่สามารถทำงานร่วมกับ Zest เพื่อสร้างข้อมูลทดสอบและนำ Feedback จากการทำงานของโปรแกรมมาใช้ในการค้นหาข้อมูลทดสอบเพิ่มเติม

2.7 ปัญญาประดิษฐ์และ Large Language Models

ปัญญาประดิษฐ์ (Artificial Intelligence: AI) เป็นเทคโนโลยีที่ช่วยให้ระบบคอมพิวเตอร์สามารถประมวลผลข้อมูลและสร้างผลลัพธ์ที่มีลักษณะคล้ายการทำงานของมนุษย์ สำหรับการทดสอบซอฟต์แวร์ Large Language Models (LLMs) สามารถนำมาใช้สร้าง Unit Test จาก Source Code หรือข้อมูลที่อธิบายการทำงานของโปรแกรมได้

บทที่ 3

วิธีการดำเนินงาน

1. รูปแบบการทดลอง

โครงการนี้เป็นทดลองเชิงเปรียบเทียบ โดยนำเทคนิคการสร้างชุดทดสอบที่แตกต่างกันมาทดลองกับโปรแกรมและข้อบกพร่องจาก Defects4J ภายใต้เงื่อนไขที่กำหนด

เทคนิคที่นำมาเปรียบเทียบประกอบด้วย

1.1 Grey Wolf Optimizer

เป็น nature-inspired metaheuristic ที่ Mirjalili et al. (2014) เสนอขึ้น โดยจำลองพฤติกรรม การล่าเหยื่อและลำดับชั้นทางสังคมในฝูงหมาป่า สมาชิกแบ่งออกเป็น 4 ระดับ ดังนี้

- 1.1.1 Alpha (α) คำตอบที่ดีที่สุด (ทำหน้าที่เหมือนผู้นำ นำทางการค้นหาหลัก)
- 1.1.2 Beta (β) คำตอบที่ดีที่สุดอันดับ 2 (ช่วย α ตัดสินใจ/นำทาง)
- 1.1.3 Delta (δ) คำตอบที่ดีที่สุดอันดับ 3 (สนับสนุนการค้นหา)
- 1.1.4 Omega (ω) คำตอบที่เหลือทั้งหมด ซึ่งจะปรับตำแหน่งตาม α, β, δ

1.2 Zest

เป็นอัลกอริทึม Coverage-Guided Fuzzing ที่พัฒนาต่อยอดจากอัลกอริทึม Fuzz testing ทั่วไป โดยมีแนวคิดสำคัญคือ การสร้างข้อมูลนำเข้าที่มีโครงสร้างผ่าน Parametric Generator และใช้ Code Coverage ร่วมกับ Semantic Validity เป็นข้อมูลย้อนกลับเพื่อคัดเลือกข้อมูลนำเข้าที่มีประโยชน์สำหรับการสร้างกรณีทดสอบใหม่ มีแนวคิดหลัก ได้แก่

- 1.2.1 Syntax parsing stage แปลง raw input เป็นโครงสร้างข้อมูล (เช่น AST)
- 1.2.2 Semantic analysis stage ตรวจสอบเงื่อนไขเชิงความหมาย และรันตรรกะหลักของโปรแกรม

1.3 การสร้างชุดทดสอบด้วย Large Language Model

การสร้างชุดทดสอบด้วย Large Language Model (LLM) เป็นแนวทางการสร้างกรณีทดสอบอัตโนมัติโดยใช้ความสามารถของโมเดลภาษาขนาดใหญ่ในการวิเคราะห์โครงสร้างและพฤติกรรมของโปรแกรม แล้วสร้างโค้ดทดสอบสำหรับตรวจสอบการทำงานของโปรแกรม ในการทดลองนี้เลือกใช้ LLM จำนวน 2 โมเดล ได้แก่ Gemini 3.7 Flash และ DeepSeek V4 Flash เพื่อสร้างชุดทดสอบสำหรับโปรแกรมและข้อบกพร่องจาก Defects4J

2. เครื่องมือและสภาพแวดล้อม

ตารางที่ 1 เครื่องมือและสภาพแวดล้อม

รายการ	รายละเอียด
Operating System	Ubuntu บน WSL2
Benchmark	Defects4J
Test Framework	JUnit
Search-based Tool	EvoSuite
Fuzzing Tool	Zest / JQF
AI Models	Gemini และ DeepSeek

จากตารางที่ 1 แสดงเครื่องมือและสภาพแวดล้อมที่ใช้ในการทดลอง ซึ่งดำเนินการบนระบบปฏิบัติการ Ubuntu ที่ติดตั้งผ่าน WSL2 (Windows Subsystem for Linux 2) ทำให้สามารถใช้งานสภาพแวดล้อมแบบ Linux บนเครื่อง Windows ได้โดยไม่ต้องติดตั้งระบบปฏิบัติการแยก และรองรับเครื่องมือทดสอบซอฟต์แวร์ที่พัฒนามาสำหรับ Linux เป็นหลัก

ชุดข้อมูลที่ใช้คือ Defects4J ซึ่งเป็นฐานข้อมูลข้อบกพร่องจริง (real bugs) ที่รวบรวมมาจากโครงการโอเพนซอร์สภาษา Java โดยแต่ละข้อบกพร่องมีทั้งเวอร์ชันที่มีข้อบกพร่องและเวอร์ชันที่ได้รับการแก้ไขแล้ว จึงเหมาะสำหรับใช้เป็นมาตรฐานในการประเมินประสิทธิภาพของกรณีทดสอบที่สร้างขึ้นว่าสามารถตรวจพบข้อบกพร่องได้หรือไม่ ส่วนกรอบการทดสอบ (Test Framework) ใช้ JUnit ซึ่งเป็นเฟรมเวิร์กมาตรฐานสำหรับเขียนและรันการทดสอบหน่วย (unit test) ในภาษา Java

สำหรับการสร้างกรณีทดสอบอัตโนมัติ ได้เลือกใช้เครื่องมือ 3 แนวทาง ได้แก่ (1) EvoSuite ซึ่งเป็นเครื่องมือแบบ search-based ที่ใช้อัลกอริทึมเชิงพันธุกรรม (genetic algorithm) ในการค้นหาชุดกรณีทดสอบที่ครอบคลุมโค้ดให้มากที่สุด และสร้างผลลัพธ์ออกมาเป็นคลาสทดสอบ JUnit (2) Zest/JQF ซึ่งเป็นเครื่องมือ fuzzing แบบ coverage-guided สำหรับภาษา Java โดย JQF เป็นแพลตฟอร์มหลัก และ Zest เป็นเทคนิคที่สร้างข้อมูลนำเข้าที่มีโครงสร้างถูกต้องตามรูปแบบที่โปรแกรมต้องการ เพื่อเข้าถึงส่วนลึกของตรรกะโปรแกรมได้ดีขึ้น และ (3) โมเดลปัญญาประดิษฐ์ ได้แก่ Gemini และ DeepSeek ซึ่งเป็นโมเดลภาษาขนาดใหญ่ (Large Language Model) ที่นำมาใช้สร้างกรณีทดสอบจากการวิเคราะห์ซอร์สโค้ด

การเลือกเครื่องมือที่มีแนวทางแตกต่างกันนี้ ทำให้สามารถเปรียบเทียบประสิทธิภาพของการสร้างกรณีทดสอบอัตโนมัติทั้งแบบดั้งเดิม (search-based และ fuzzing) กับแบบที่ใช้ปัญญาประดิษฐ์ ภายใต้ชุดข้อมูลและสภาพแวดล้อมเดียวกันได้อย่างเป็นธรรม

3. ชุดข้อมูล Defects4J

Defects4J เป็นชุดข้อมูล Benchmark ที่ใช้สำหรับการวิจัยด้านการทดสอบและการวิเคราะห์โปรแกรมภาษา Java โดยประกอบโปรเจคซอฟต์แวร์ที่มีข้อบกพร่องจริงและข้อมูลของเวอร์ชันก่อนและหลังการแก้ไขข้อบกพร่อง

สำหรับแต่ละกรณีทดสอบ จะมีการบันทึกข้อมูลเช่น

- 3.1 ชื่อโปรเจค
- 3.2 หมายเลข Bug
- 3.3 ค่า Coverage

4. ขั้นตอนการทดลอง

- 4.1 เลือกโปรเจคและ Bug ID จาก Defects4J
- 4.2 Checkout Buggy Version และตรวจสอบสภาพแวดล้อมของโปรเจค
- 4.3 ระบุ Target Class และเตรียม Classpath ที่จำเป็น
- 4.4 สร้าง Test Case ด้วยเครื่องมือแต่ละประเภทตามการตั้งค่าที่กำหนด
- 4.5 ตรวจสอบว่า Test Case สามารถ Compile ได้หรือไม่
- 4.6 นำ Test Case ที่ผ่านการ Compile ไปทดสอบและเก็บข้อมูล Coverage
- 4.7 รวบรวมข้อมูลลง CSV และตรวจสอบความครบถ้วนก่อนวิเคราะห์

5. กระบวนการการทำงาน

5.1 กระบวนการสร้างชุดทดสอบด้วยปัญญาประดิษฐ์ (AI-Based Test Generation Process)

ในการสร้างชุดทดสอบด้วยแบบจำลองปัญญาประดิษฐ์ (DeepSeek-v4-flash ผ่าน KGU IntelSphere API) ผู้วิจัยพบว่าการสร้างชุดทดสอบจากคำสั่ง (Prompt) เพียงครั้งเดียวไม่สามารถสร้างชุดทดสอบที่คอมไพล์ผ่านได้ในทุกกรณี เนื่องจากแบบจำลองมีแนวโน้มที่จะสร้างข้อมูลเท็จ (Hallucination) เกี่ยวกับโครงสร้างของโปรแกรม เช่น การอ้างอิง Class หรือ Method ที่ไม่มีอยู่จริงใน Source Code ที่ให้ไป ผู้วิจัยจึงออกแบบกระบวนการทำงานเป็นสองส่วนหลัก ดังนี้

5.1.1 การปรับปรุง Prompt แบบวนซ้ำ (Iterative Prompt Refinement)

หลังจากตรวจพบรูปแบบข้อผิดพลาดที่เกิดขึ้นซ้ำๆ จากการคอมไพล์ชุดทดสอบที่สร้างโดยแบบจำลอง ผู้วิจัยได้วิเคราะห์ Log ข้อผิดพลาดและเพิ่มข้อกำหนดเฉพาะ (Guidance) เข้าไปใน Prompt ต้นฉบับ เพื่อป้องกันไม่ให้เกิดข้อผิดพลาดแบบเดิมซ้ำอีกในการสร้างชุดทดสอบครั้งต่อไป การเพิ่มข้อกำหนดดังกล่าวถูกออกแบบให้เป็นแบบ Idempotent กล่าวคือ สคริปต์จะตรวจสอบ Marker เฉพาะในไฟล์ Prompt ก่อนทุกครั้ง หากพบว่ามีการกำหนดนั้นอยู่แล้วจะไม่แทรกซ้ำ เพื่อป้องกันการขยายขนาดของ Prompt โดยไม่จำเป็น

นอกจากนี้ ระบบยังมีการกำหนดจำนวนรอบของการแก้ไขตัวเอง (Self-Refinement) ไว้ไม่เกิน 1 รอบ (MAX_REFINEMENT_ROUNDS = 1) กล่าวคือ หากชุดทดสอบที่สร้างจาก Prompt ครั้งแรกไม่สามารถคอมไพล์ผ่านได้ ระบบจะส่งข้อความแจ้งข้อผิดพลาดจากการคอมไพล์ (Compile Error) กลับไปให้แบบจำลองแก้ไขอีกครั้ง ก่อนที่จะบันทึกผลเป็นความล้มเหลว (compile_ok = False) ในกรณีที่ยังไม่ผ่าน

5.1.2 ข้อจำกัดด้านปริมาณการใช้งาน Token (Token Quota)

เนื่องจาก KGU IntelSphere API มีการกำหนดปริมาณการใช้งาน Token สูงสุดต่อวัน (Daily Token Limit) ไว้ที่ 1,000,000 Token ต่อแบบจำลองต่อวัน การสร้างชุดทดสอบสำหรับโครงการที่มีจำนวนข้อบกพร่อง (Bug) จำนวนมาก เช่น Math (106 ข้อบกพร่อง) หรือ Mockito (38 ข้อบกพร่อง) จึงอาจไม่สามารถดำเนินการให้เสร็จสมบูรณ์ภายในวันเดียว กระบวนการทดลองจึงต้องหยุดชั่วคราวเมื่อ API ตอบกลับด้วยสถานะแจ้งว่าถึงขีดจำกัดการใช้งานประจำวัน และดำเนินการต่อในวันถัดไปเมื่อปริมาณ Token ถูกรีเซ็ต ข้อจำกัดนี้ถือเป็นปัจจัยภายนอกที่ส่งผลกระทบต่อระยะเวลาในการเก็บข้อมูล มิใช่ข้อบ่งชี้ถึงความสามารถของแบบจำลองในการสร้างชุดทดสอบ

6. ตัวชี้วัดที่ใช้ในการประเมินผล

6.1 Line Coverage สัดส่วนของบรรทัดคำสั่งที่ถูกเรียกใช้งานระหว่างการทดสอบ

Line Coverage คือสัดส่วนของบรรทัดคำสั่งที่สามารถทำงานได้ (executable lines) ซึ่งถูกเรียกใช้งานอย่างน้อยหนึ่งครั้งระหว่างการทดสอบ เมื่อเทียบกับจำนวนบรรทัดคำสั่งที่สามารถทำงานได้ทั้งหมด โดยไม่นับบรรทัดว่าง บรรทัดความคิดเห็น (comment) และบรรทัดประกาศที่ไม่มีการทำงาน คำนวณได้จากสมการ

$$\text{Line Coverage} = \frac{\text{Covered Lines}}{\text{Total Lines}} \times 100$$

6.2 Branch Coverage สัดส่วนของ Branch ที่ถูกทดสอบให้ทำงาน

Branch Coverage หรือ Decision Coverage คือสัดส่วนของกิ่ง (branch) ที่ถูกทดสอบให้ทำงาน โดยกิ่งหมายถึงผลลัพธ์ที่เป็นไปได้ของจุดตัดสินใจในโปรแกรม เช่น if, switch, while และ for ซึ่งแต่ละจุดตัดสินใจจะมีอย่างน้อยสองกิ่ง คือกรณีที่ผลเป็นจริง (true) และเป็นเท็จ (false) คำนวณได้จากสมการ

$$\text{Branch Coverage} = \frac{\text{Covered Branches}}{\text{Total Branches}} \times 100$$

6.3 Condition Coverage สัดส่วนของ Condition ที่ถูกทดสอบให้ทำงาน

Condition Coverage คือสัดส่วนของเงื่อนไขย่อย (atomic condition) ภายในนิพจน์ตัดสินใจ ที่ถูกทดสอบให้มีผลเป็นทั้งจริงและเท็จ ตัวอย่างเช่น นิพจน์ $\text{if } (a > 0 \ \&\& \ b > 0)$ ประกอบด้วยเงื่อนไขย่อย 2 เงื่อนไข คือ $a > 0$ และ $b > 0$ ซึ่งต้องทดสอบให้แต่ละเงื่อนไขเป็นทั้งจริงและเท็จอย่างน้อยหนึ่งครั้ง คำนวณได้จากสมการ

$$\text{Condition Coverage} = \frac{\text{Covered Conditions}}{\text{Total Conditions}} \times 100$$

บทที่ 4

ผลการทดลองและการวิเคราะห์

1. ภาพรวมการทดลอง

ตารางที่ 2 จำนวน Bug ที่สามารถทดสอบได้ด้วย GWO, Zest, Gemini และ DeepSeek จำแนกตามโครงการ Defects4J

Project	จำนวน Bug ทั้งหมด	GWO	Zest	Gemini	DeepSeek
Chart	26	17	26	15	17
Cli	39	38	39	29	18
Closure	174	8	131	36	8
Codec	18	17	18	16	15
Collections	28	23	28	23	15
Compress	47	32	47	34	34
Csv	16	10	16	12	7
Gson	18	6	18	9	11
JacksonCore	26	18	26	11	9
JacksonDatabind	110	4	110	21	17
JacksonXml	6	0	6	3	3
Jsoup	93	32	93	64	69
JXPath	22	0	22	13	14
Lang	61	49	61	51	46
Math	106	101	105	77	69
Mockito	38	18	38	17	35
Time	26	0	25	16	14
Total	854	373	809	447	273

จากตารางที่ 2 การศึกษาครอบคลุม Defects4J จำนวน 17 โครงการ รวมทั้งสิ้น 854 Bug โดยจำนวน Bug ที่สามารถนำมาดำเนินการทดลองและได้รับผลลัพธ์จากแต่ละเครื่องมือมีความแตกต่างกัน ได้แก่ GWO, Zest, Gemini-3.7-flash และ DeepSeek-v4-flash โดย GWO สามารถดำเนินการทดลองได้ 373 Bug, Zest 809 Bug, Gemini-3.7-flash 447 Bug และ DeepSeek-v4-flash 273 Bug ตามลำดับ

ความแตกต่างของจำนวน Bug ที่ได้รับผลการทดลองเกิดจากข้อจำกัดและเงื่อนไขในการทำงานของแต่ละเครื่องมือ เช่น การสร้าง Test Suite ไม่สำเร็จ การ Compile ไม่ผ่าน หรือไม่สามารถดำเนินการวัด Coverage ได้ ส่งผลให้จำนวน Bug ที่มีข้อมูลผลการทดลองสำหรับแต่ละเครื่องมือไม่เท่ากัน

ดังนั้น ในขั้นตอนการวิเคราะห์ผลการทดลอง จะพิจารณาเฉพาะ Bug ที่มีข้อมูล Line Coverage ที่สามารถตรวจสอบได้และสามารถนำไปคำนวณได้จริง โดยจะไม่นำกรณีที่ไม่มีข้อมูล Line Coverage หรือไม่สามารถประเมิน Coverage ได้มารวมในการคำนวณค่าเฉลี่ย ทั้งนี้ เพื่อให้ข้อมูลที่ใช้ในการเปรียบเทียบระหว่างเครื่องมือมีความสอดคล้องกัน และลดผลกระทบจากกรณีที่ไม่สามารถสร้าง Compile หรือประเมิน Test Suite ได้

2. ผลด้าน Line Coverage

2.1 ค่าเฉลี่ยผลด้าน Line Coverage สำหรับทุกบัคที่แต่ละวิธีทำได้แยกเป็นโปรเจค

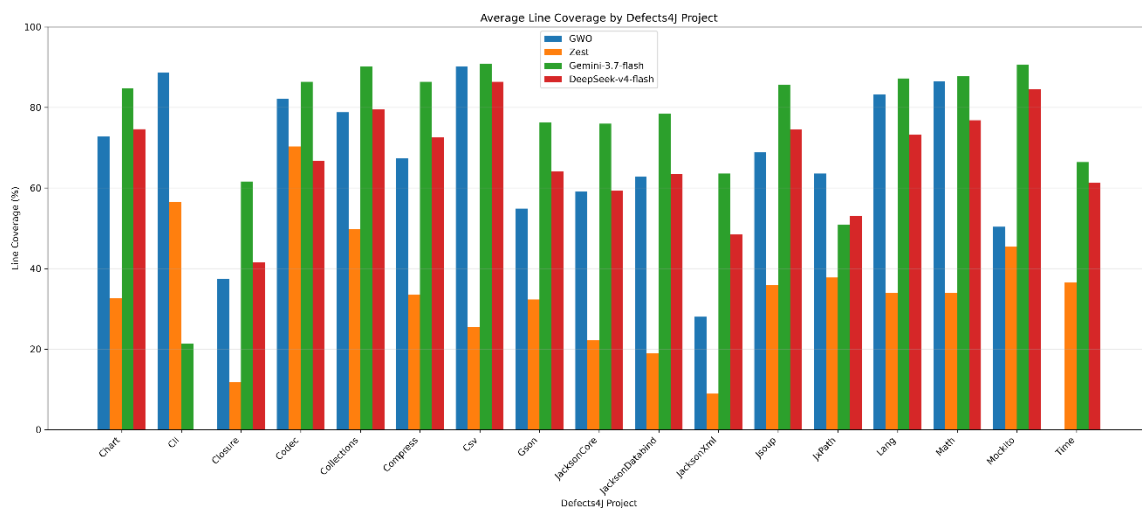
ตารางที่ 3 ค่าเฉลี่ย Line Coverage จำแนกตามโครงการ Defects4J และวิธีการทดสอบ

Project	GWO	Zest	Gemini	DeepSeek
Chart	72.78%	32.58%	84.06%	73.66%
Cli	88.68%	56.58%	20.79%	0.00%
Closure	37.41%	11.86%	62.36%	41.55%
Codec	82.15%	70.29%	88.36%	68.12%
Collections	78.80%	49.80%	90.08%	79.47%
Compress	67.42%	33.52%	85.19%	72.44%
Csv	90.11%	25.48%	90.74%	86.38%
Gson	56.88%	32.34%	78.36%	64.19%
JacksonCore	59.19%	22.17%	75.96%	58.01%
JacksonDatabind	63.90%	19.04%	78.37%	63.53%
JacksonXml	28.00%	8.90%	63.60%	48.53%

Jsoup	68.83%	35.92%	86.29%	74.69%
JXPath	63.59%	37.79%	46.62%	47.46%
Lang	83.24%	33.92%	87.24%	73.18%
Math	86.40%	34.05%	88.10%	77.30%
Mockito	50.54%	45.45%	90.58%	82.54%
Time	N/A	36.58%	67.52%	61.36%

จากตารางที่ 3 พบว่าค่า *Line Coverage* มีความแตกต่างกันทั้งระหว่างเครื่องมือและระหว่างโครงการ โดย *Gemini-3.7-flash* มีค่า *Line Coverage* อยู่ในระดับสูงในหลายโครงการ เช่น *Collections*, *Compress*, *Gson*, *JacksonCore*, *JacksonDatabind*, *JacksonXml*, *Jsoup*, *Lang* และ *Mockito* ขณะที่ *GWO* มีค่า *Line Coverage* สูงในบางโครงการ เช่น *Cli*, *Codec*, *Csv* และ *JXPath* เมื่อพิจารณาจากค่าที่ปรากฏในแต่ละโครงการ

สำหรับ *Zest* ค่า *Line Coverage* มีความแตกต่างกันตามโครงการ โดยมีค่าตั้งแต่ 8.90% ใน *JacksonXml* ถึง 70.29% ใน *Codec* ส่วน *DeepSeek-v4-flash* มีค่า *Line Coverage* ตั้งแต่ 41.55% ใน *Closure* ถึง 86.38% ใน *Csv*



ภาพที่ 1 ค่าเฉลี่ย Line Coverage จำแนกตามโครงการ Defects4J และวิธีการทดสอบ

2.2 ค่าเฉลี่ยผลด้าน *Line Coverage* เฉพาะบัคที่ทุกวิธีสามารถสร้าง *Test Case* ได้เหมือนกัน

ตารางที่ 4 ค่าเฉลี่ย Line Coverage ของ Common Bugs จำแนกตามโครงการ Defects4J และวิธีการทดสอบ

Project	Common Bugs	GWO	Zest	Gemini	DeepSeek
Chart	13	82.98%	43.83%	84.30%	74.18%
Cli	0	N/A	N/A	N/A	N/A
Closure	2	5.23%	5.80%	89.35%	46.24%
Codec	12	88.33%	70.56%	86.02%	87.11%
Collections	13	82.26%	57.64%	93.83%	80.09%
Compress	23	75.34%	42.72%	87.71%	78.41%
Csv	5	93.54%	46.24%	97.33%	94.09%
Gson	6	72.06%	32.77%	70.48%	67.07%
JacksonCore	4	85.50%	58.79%	91.65%	82.35%
JacksonDatabind	2	94.87%	41.93%	84.38%	71.05%
JacksonXml	2	20.54%	2.01%	58.43%	52.21%
Jsoup	51	79.75%	43.24%	87.60%	82.09%
JXPath	10	65.84%	27.88%	48.28%	43.72%
Lang	37	83.20%	36.77%	90.63%	74.31%
Math	55	91.39%	37.92%	87.75%	78.70%
Mockito	14	47.10%	42.62%	88.70%	86.14%
Time	0	N/A	N/A	N/A	N/A

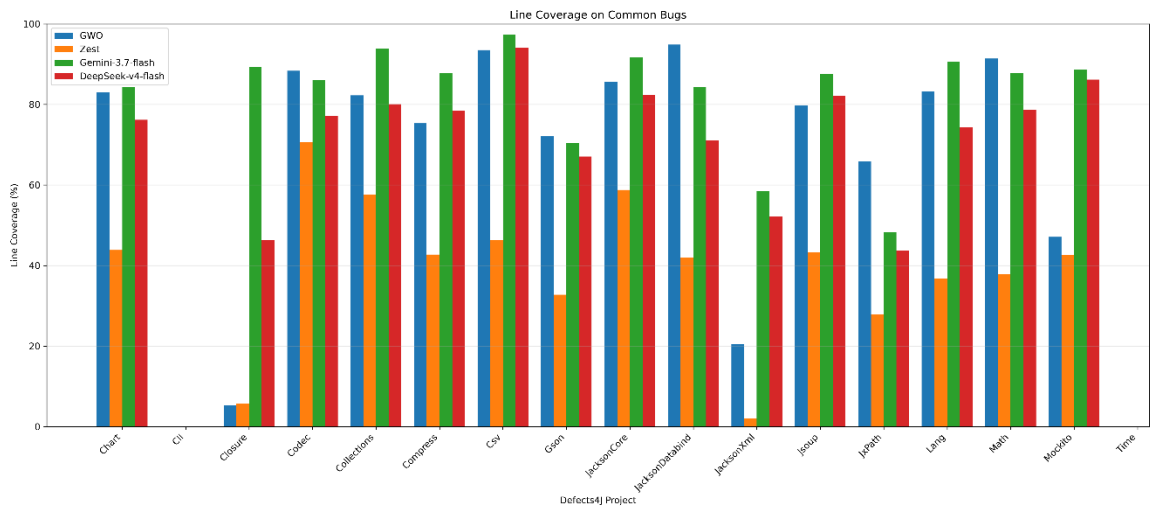
จากตารางที่ 4 พบว่า จำนวน Common Bugs แตกต่างกันอย่างมากในแต่ละโครงการ โดย Jsoup มีจำนวน Common Bugs มากที่สุด 51 บั๊ก รองลงมาคือ Math 55 บั๊ก, Lang 37 บั๊ก และ Compress 23 บั๊ก ขณะที่ Cli และ Time ไม่มี Common Bugs จึงไม่สามารถคำนวณ Line Coverage ในกลุ่มดังกล่าวได้

สำหรับโปรเจกต์ที่มี Common Bugs ผล Line Coverage ของแต่ละวิธีมีความแตกต่างกันในหลายโปรเจกต์ โดย Gemini-3.7-flash มีค่า Line Coverage สูงในหลายโครงการ เช่น Csv 97.33%, Collections 93.83%, Lang 90.63%, Mockito 88.70% และ Jsoup 87.60% ขณะที่ GWO มีค่า Line Coverage สูงในบางโปรเจกต์ เช่น Math 91.39%, Csv 93.54%, JacksonDatabind 94.87% และ Codec 88.33%

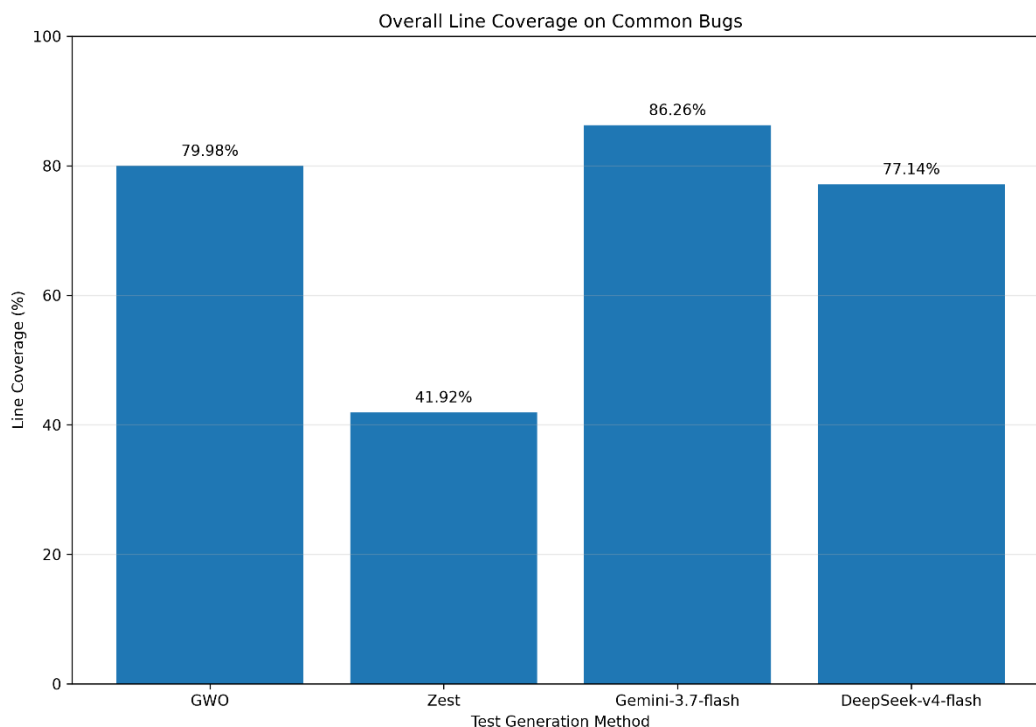
ส่วน Zest มีค่า Line Coverage ต่ำกว่าอีกสามวิธีในโครงการส่วนใหญ่ โดยค่าที่สูงที่สุดในกลุ่ม Common Bugs คือ Csv 46.24% และ Closure 5.80% เป็นค่าที่ต่ำมากเมื่อเทียบกับวิธีอื่นในโครงการเดียวกัน ส่วน DeepSeek-v4-flash มีค่าอยู่ระหว่าง GWO และ Gemini ในหลายโครงการ เช่น Collections 80.09%, Compress 78.41%, Jsoup 82.09% และ Mockito 86.14%

เมื่อพิจารณาเป็นรายโปรเจกต์ สามารถสังเกตได้ว่า Csv เป็นโครงการที่ทุกวิธีมี Line Coverage ค่อนข้างสูง โดย GWO, Gemini และ DeepSeek มีค่าเกิน 90% ขณะที่ Zest อยู่ที่ 46.24% ในทางกลับกัน JacksonXml มีค่า Line Coverage ต่ำกว่าวิธีอื่นอย่างชัดเจน โดยเฉพาะ Zest ที่มีค่าเพียง 2.01% นอกจากนี้ Closure แสดงความแตกต่างระหว่างวิธีค่อนข้างมาก โดย GWO มีค่า 5.23% ในขณะที่ Gemini มีค่า 89.35%

สำหรับโครงการขนาดใหญ่ที่มี Common Bugs จำนวนมาก เช่น Jsoup และ Math ผลการทดลองยังแสดงให้เห็นความแตกต่างของ Line Coverage ระหว่างวิธีอย่างชัดเจน โดย Math มีค่า GWO 91.39%, Zest 37.92%, Gemini 87.75% และ DeepSeek 78.70% ส่วน Jsoup มีค่า GWO 79.75%, Zest 43.24%, Gemini 87.60% และ DeepSeek 82.09%



ภาพที่ 2 กราฟค่าเฉลี่ย Line Coverage ของ Common Bugs จำแนกตามโครงการ Defects4J และวิธีการทดสอบ



ภาพที่ 3 กราฟแสดงภาพรวม Line Coverage ของแต่ละวิธี

จากภาพที่ 3 แสดงการเปรียบเทียบค่า *Overall Line Coverage* ของเครื่องมือสร้างชุดทดสอบทั้ง 4 วิธี ได้แก่ GWO, Zest, Gemini-3.7-flash และ DeepSeek-v4-flash โดยพิจารณาเฉพาะ *Common Bugs* จำนวน 249 บั๊ก ซึ่งเป็นบั๊กที่ทั้ง 4 วิธีมีข้อมูล *Line Coverage* ที่สามารถนำมาเปรียบเทียบกันได้

จากผลการทดลองพบว่า Gemini-3.7-flash มีค่า *Overall Line Coverage* เท่ากับ 86.26% รองลงมาคือ GWO เท่ากับ 79.98%, DeepSeek-v4-flash เท่ากับ 77.14% และ Zest เท่ากับ 41.92% ตามลำดับ โดยค่าเหล่านี้คำนวณจาก *Line Coverage* ของ *Common Bugs* ทั้ง 249 บั๊ก เพื่อให้ทุกวิธีถูกเปรียบเทียบภายใต้ชุดบั๊กเดียวกัน

เมื่อพิจารณาความแตกต่างระหว่างวิธี พบว่า Gemini-3.7-flash มีค่า *Line Coverage* สูงกว่า GWO ประมาณ 6.28 จุดเปอร์เซ็นต์ และสูงกว่า DeepSeek-v4-flash ประมาณ 9.12 จุดเปอร์เซ็นต์ ขณะที่ Zest มีค่าต่ำกว่าวิธีอื่นอย่างชัดเจน โดยต่ำกว่า Gemini-3.7-flash ประมาณ 44.34 จุดเปอร์เซ็นต์ ผลดังกล่าวสะท้อนให้เห็นว่า ในชุด *Common Bugs* ที่ใช้สำหรับการเปรียบเทียบ Gemini-3.7-flash สามารถสร้างชุดทดสอบที่ครอบคลุมบรรทัดของโค้ดได้ในระดับสูง ขณะที่ GWO และ DeepSeek-v4-flash มีค่า *Coverage* อยู่ในระดับใกล้เคียงกันมากกว่า ส่วน Zest มีค่า *Overall Line Coverage* ต่ำกว่าอีกสามวิธีอย่างเห็นได้ชัด

3. ผลด้าน Branch และ Condition Coverage

3.1 Branch Coverage

ตารางที่ 5 ค่าเฉลี่ย Branch Coverage ของ GWO และ Zest จำแนกตามโปรเจค Defects4J และ จำนวน Common Bugs

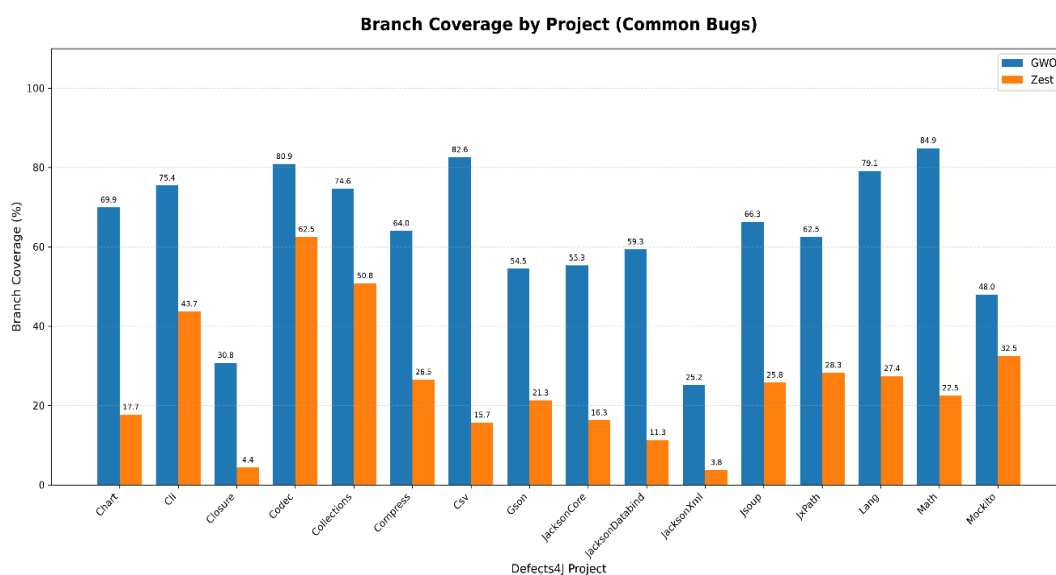
Project	GWO	Zest	Common Bugs
Chart	69.92%	17.68%	24
Cli	83.85%	43.69%	6
Closure	33.10%	4.36%	130
Codec	77.84%	62.48%	17
Collections	74.64%	50.78%	25
Compress	64.00%	26.53%	46
Csv	82.56%	15.70%	16
Gson	54.46%	21.29%	17
JacksonCore	55.31%	16.28%	25
JacksonDatabind	59.34%	11.30%	78
JacksonXml	25.17%	3.75%	6
Jsoup	66.80%	25.75%	90
JXPath	62.48%	28.26%	22
Lang	79.12%	27.43%	56
Math	N/A	22.50%	-
Mockito	47.99%	32.48%	32
Time	N/A	N/A	-

จากตารางที่ 5 แสดงการเปรียบเทียบค่าเฉลี่ย Branch Coverage ระหว่างวิธี Grey Wolf Optimizer (GWO) และ Zest จำแนกตาม Project โดยพิจารณาเฉพาะ Bug ที่มีผลการทดลองจากทั้งสองวิธีและสามารถนำมาเปรียบเทียบกันได้ ซึ่งจำนวนดังกล่าวแสดงในคอลัมน์ Common Bugs ผลการทดลองพบว่า ค่า Branch Coverage ของ GWO มีความแตกต่างกันในแต่ละ Project โดยมีค่าตั้งแต่ 25.17% ใน Project JacksonXml ไปจนถึง 83.85% ใน Project Cli ขณะที่ Zest มีค่า

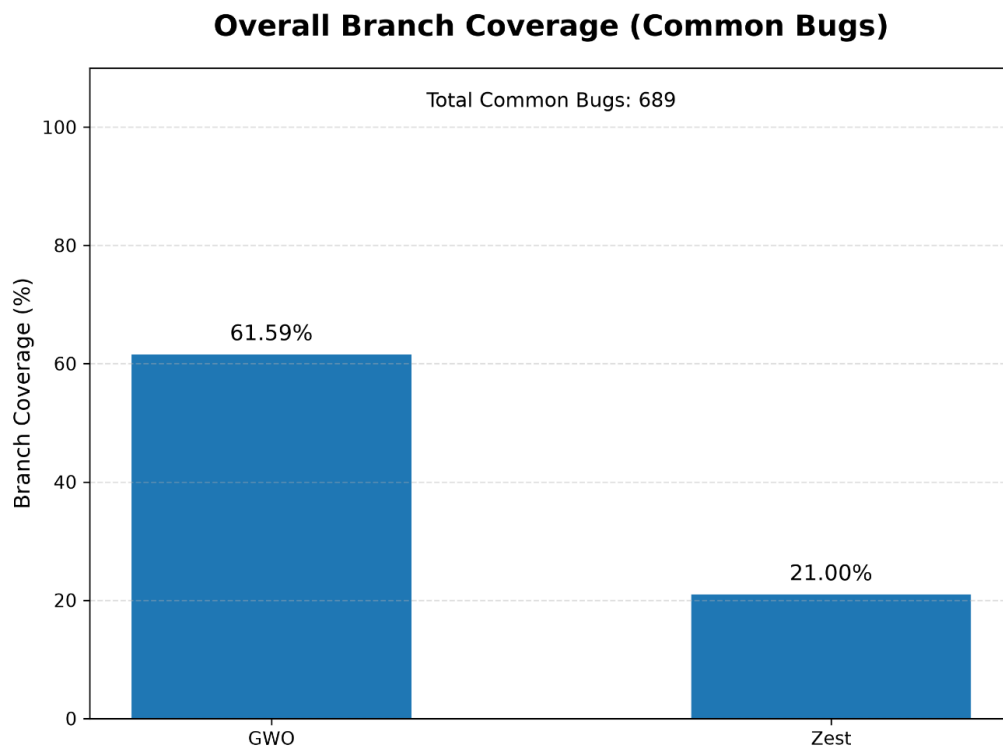
Branch Coverage อยู่ในช่วง 3.75% ถึง 62.48% โดย Project Codec มีค่า Branch Coverage ของ Zest สูงที่สุดในกลุ่มที่แสดงในตาราง และ Project JacksonXml มีค่าต่ำที่สุด

เมื่อพิจารณาเป็นราย Project พบว่า GWO มีค่า Branch Coverage สูงกว่า Zest ในหลายโครงการ เช่น Chart, Cli, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang และ Mockito ในขณะที่ Zest มีค่า Branch Coverage สูงกว่า GWO ใน Project Codec และ Project Closure ตามข้อมูลที่ปรากฏในตาราง

สำหรับ Project Math และ Time ไม่สามารถแสดงค่า Branch Coverage ของทั้งสองวิธีได้ครบถ้วน จึงระบุเป็น N/A และไม่รวมในการเปรียบเทียบเชิงตัวเลข



ภาพที่ 4 กราฟแสดงการเปรียบเทียบ Branch Coverage ของ GWO และ Zest จำแนกตามโปรเจกต์



ภาพที่ 5 กราฟแสดงการเปรียบเทียบภาพรวมของ Branch Coverage ระหว่าง GWO และ Zest

3.2 Condition Coverage

ตารางที่ 6 ค่าเฉลี่ย Condition Coverage ของ Gemini และ DeepSeek จำแนกตามโปรเจค Defects4J และจำนวน Common Bugs

Project	Gemini	DeepSeek	Common Bugs
Chart	35.84%	31.62%	13
Cli	4.94%	0.00%	15
Closure	77.02%	38.72%	2
Codec	63.08%	56.86%	13
Collections	74.92%	53.03%	14
Compress	77.02%	69.49%	23
Csv	89.04%	86.30%	5
Gson	28.65%	35.31%	6
JacksonCore	57.25%	44.93%	4

JacksonDatabind	54.43%	45.57%	3
JacksonXml	37.16%	42.57%	2
Jsoup	64.46%	59.54%	51
JXPath	26.16%	15.15%	10
Lang	72.76%	50.82%	42
Math	70.12%	59.33%	60
Mockito	82.27%	84.55%	15
Time	52.54%	40.23%	10

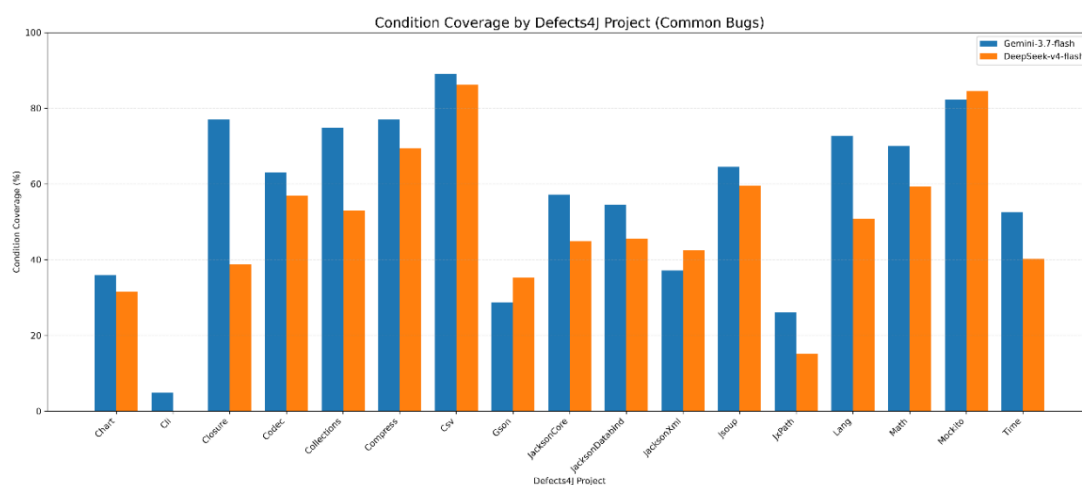
จากตารางที่ 6 แสดงการเปรียบเทียบค่า *Condition Coverage* ระหว่าง *Gemini-3.7-flash* และ *DeepSeek-v4-flash* โดยพิจารณาเฉพาะ *Common Bugs* ของแต่ละ *Defects4J Project* ซึ่งเป็นบีกที่ทั้งสองวิธีมีข้อมูล *Condition Coverage* ที่สามารถนำมาเปรียบเทียบกันได้ จากผลการทดลอง พบว่า *Common Bugs* รวมทั้งหมดมีจำนวน 288 บีก โดย *Gemini-3.7-flash* มีค่า *Overall Condition Coverage* เท่ากับ 60.03% ขณะที่ *DeepSeek-v4-flash* มีค่า 48.64% ดังนั้นค่าทั้งสองวิธีแตกต่างกัน 11.39 จุดเปอร์เซ็นต์

เมื่อพิจารณาเป็นราย *Project* พบว่า *Gemini-3.7-flash* มีค่า *Condition Coverage* สูงกว่า *DeepSeek-v4-flash* ใน 14 จาก 17 *Project* ได้แก่ *Chart*, *Cli*, *Closure*, *Codec*, *Collections*, *Compress*, *Csv*, *JacksonCore*, *JacksonDatabind*, *Jsoup*, *JXPath*, *Lang*, *Math* และ *Time* ขณะที่ *DeepSeek-v4-flash* มีค่าสูงกว่า *Gemini-3.7-flash* ใน 3 *Project* ได้แก่ *Gson*, *JacksonXml* และ *Mockito* *Project* ที่มีค่า *Condition Coverage* ของ *Gemini-3.7-flash* สูง ได้แก่ *Csv* 89.04%, *Mockito* 82.27%, *Closure* และ *Compress* 77.02% และ *Collections* 74.92% ส่วน *Project* ที่มีค่าต่ำ ได้แก่ *Cli* 4.94%, *JXPath* 26.16% และ *Gson* 28.65%

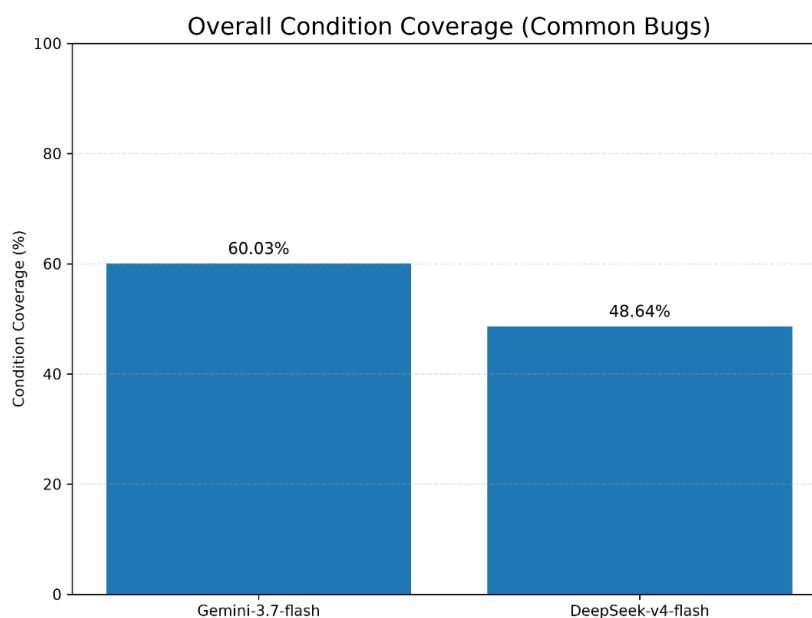
สำหรับ *DeepSeek-v4-flash* ค่า *Coverage* สูงสุดพบใน *Mockito* 84.55%, *Csv* 86.30% และ *Compress* 69.49% ขณะที่ค่าต่ำสุดพบใน *Cli* 0.00%, *JXPath* 15.15% และ *Gson* 35.31%

เมื่อพิจารณาความแตกต่างระหว่างสองวิธี พบว่า *Closure* มีช่องว่างของ *Coverage* ค่อนข้างมาก โดย *Gemini-3.7-flash* มีค่า 77.02% ขณะที่ *DeepSeek-v4-flash* มีค่า 38.72% เช่นเดียวกับ *Lang* ซึ่งมีค่า 72.76% และ 50.82% ตามลำดับ ในทางกลับกัน *Mockito* เป็น *Project* ที่ *DeepSeek-v4-flash* มีค่า *Coverage* สูงกว่า *Gemini-3.7-flash* โดยมีค่า 84.55% เทียบกับ 82.27%

ดังนั้น ผลการทดลองในตารางนี้แสดงให้เห็นถึงความแตกต่างของความสามารถในการครอบคลุมเงื่อนไขของโปรแกรมระหว่าง *Gemini-3.7-flash* และ *DeepSeek-v4-flash* โดยผลลัพธ์มีความแตกต่างกันตามลักษณะของแต่ละ *Project* และชุดบีกที่ใช้ในการทดลอง ทั้งนี้ ค่า *Overall* ที่ 60.03% และ 48.64% คำนวณจาก *Common Bugs* รวม 288 บีก เพื่อให้การเปรียบเทียบระหว่างทั้งสองวิธีอยู่บนชุดข้อมูลที่สามารถเปรียบเทียบรวมกันได้



ภาพที่ 6 กราฟแสดงการเปรียบเทียบ Condition Coverage ระหว่าง Gemini และ DeepSeek



ภาพที่ 7 กราฟแสดงภาพรวม Condition Coverage ของ Gemini และ DeepSeek

4. การวิเคราะห์สาเหตุความล้มเหลวในการสร้างชุดทดสอบด้วยปัญญาประดิษฐ์

หัวข้อนี้วิเคราะห์สาเหตุที่ชุดทดสอบซึ่งสร้างขึ้นโดยแต่ละเทคนิคไม่สามารถคอมไพล์หรือทำงานได้สำเร็จ โดยเฉพาะในส่วนของการใช้ปัญญาประดิษฐ์ซึ่งมีรูปแบบความล้มเหลวที่มีลักษณะเฉพาะแตกต่างจากแนวทางที่ใช้อัลกอริทึม

4.1 การอ้างอิง Class หรือ Package ที่ไม่มีอยู่จริง (Hallucinated Class/Package)

แบบจำลองมีแนวโน้มที่จะสมมติว่ามี Class ภายในของไลบรารีอยู่จริงโดยไม่มีหลักฐานจาก Source Code ที่ให้ไป เช่น ในโครงการ Math แบบจำลองอ้างอิง Class ชื่อ *SplitSubHyperplane* และ *SimpleBounds* ซึ่งไม่มีอยู่จริงในตำแหน่งที่ระบุ หรือในโครงการ *Mockito* ที่อ้างอิง Class ภายในอย่าง *Discrepancy*, *MockSettingsImpl* และ *VoidMethodStubbable* ซึ่งไม่มีอยู่ใน Package ที่สมมติขึ้น

4.2 การเดาจำนวนและชนิดของ Argument ผิดพลาด (Hallucinated Arity)

แบบจำลองมักเรียกใช้ Constructor หรือ Method ด้วยจำนวน Argument ที่ไม่ตรงกับ Signature จริง เช่น การเรียก *Invocation(...)* ด้วย Argument 3 ตัว ทั้งที่ Constructor จริงต้องการ 5 ตัว หรือการเรียก *convertLocalToUTC(...)* ในโครงการ *Time* ด้วย Argument 3 ตัว ทั้งที่ Method จริงรับเพียง 2 ตัว

4.3 การเดาชนิดของค่าที่คืนกลับผิดพลาด (Hallucinated Return Type)

พบกรณีที่แบบจำลองสมมติว่า Method คืนค่าเป็น Array ทั้งที่ค่าจริงที่คืนกลับเป็น *List<T>* เช่น Method ของ Class *Complex* ในโครงการ *Math* ที่คืนค่าเป็น *List<Complex>* แต่แบบจำลองเขียน Code โดยสมมติว่าคืนค่าเป็น *Complex[]*

4.4 การ Override Method แบบไม่ครบถ้วนใน Anonymous/Inner Class

เมื่อต้องสร้าง Anonymous Class ที่ Implement มากกว่าหนึ่ง Interface พร้อมกัน แบบจำลองมัก Override Abstract Method ไม่ครบทุกตัวจากทุก Interface ที่เกี่ยวข้อง โดยเฉพาะในโครงการ *Mockito* ที่มีการใช้ Interface ซ้อนกันหลายชั้น เช่น *VerificationMode*, *OngoingInjector* และ *ContainsExtraTypeInfo*

ผลการวิเคราะห์ข้างต้นแสดงให้เห็นว่า อัตราความสำเร็จในการคอมไพล์ของชุดทดสอบที่สร้างโดย AI มีความสัมพันธ์กับความซับซ้อนของโครงสร้างภายในของโปรแกรมต้นทาง (Source Code) มากกว่าความซับซ้อนของ Bug เพียงอย่างเดียว กล่าวคือ โครงการที่มีการใช้งาน Library ภายในที่ซับซ้อน เช่น *Mockito* หรือมีจำนวน Class ที่เกี่ยวข้องกันมาก เช่น *Math* มีอัตราความล้มเหลวในการคอมไพล์สูงกว่าโครงการที่มีโครงสร้างไม่ซับซ้อน แม้ว่าผู้วิจัยจะดำเนินการปรับปรุง Prompt อย่างต่อเนื่องเพื่อป้องกันข้อผิดพลาดที่เคยเกิดขึ้นแล้วก็ตาม

5. ผลด้าน Test Suite Characteristics

5.1 Grey Wolf Optimizer

ตารางที่ 7 ค่าเฉลี่ยคุณลักษณะของชุดกรณีทดสอบที่สร้างโดย Grey Wolf Optimizer จำแนกตามโครงการ Defects4J

Project	Test Cases	Suite Length	Search Time
Chart	100.08	521.58	193.58s
Cli	37.47	215.89	170.20s
Closure	47.67	268.97	144.82s
Codec	113.28	202.76	121.59s
Collections	116.04	214.92	121.68s
Compress	47.59	149.09	122.30s
Csv	82.41	310.76	121.65s
Gson	42.55	159.55	122.10s
JacksonCore	85.56	535.62	121.88s
JacksonDatabind	54.03	315.05	132.69s
JacksonXml	27.00	228.00	122.33s
Jsoup	49.67	299.65	170.29s
JXPath	38.71	193.34	122.14s
Lang	154.68	294.62	121.59s
Math	63.27	250.42	203.39s
Mockito	8.72	31.97	188.38s
Time	N/A	N/A	121.17s

จากตารางที่ 7 ผลการทดลองด้าน *Test Suite Characteristics* ของ Grey Wolf Optimizer (GWO) พิจารณาจากจำนวน *Test Cases*, *Suite Length* และ *Search Time* ของแต่ละ Defects4J Project พบว่าคุณลักษณะของ *Test Suite* มีความแตกต่างกันตามลักษณะและความซับซ้อนของแต่ละ Project ในด้าน จำนวน *Test Cases* พบว่า *Lang* มีจำนวน *Test Cases* เฉลี่ยสูงที่สุดที่ 154.68 *Test Cases* รองลงมาคือ *Codec* ที่ 113.28 *Test Cases* และ *Collections* ที่ 116.04 *Test Cases* ขณะที่ *Mockito* มีจำนวน *Test Cases* เฉลี่ยต่ำที่สุดที่ 8.72 *Test Cases* สำหรับ *Time* ไม่สามารถรายงานจำนวน *Test Cases* และ *Suite Length* ได้ เนื่องจากไม่มีข้อมูล *Test Suite* ที่สามารถนำมาคำนวณได้

สำหรับ *Suite Length* ซึ่งสะท้อนจำนวน *Statement* โดยรวมภายใน *Test Suite* พบว่า *JacksonCore* มีค่าเฉลี่ยสูงที่สุดที่ 535.62 รองลงมาคือ *Chart* ที่ 521.58 และ *JacksonDatabind* ที่

315.05 ในขณะที่ Mockito มีค่าเฉลี่ยต่ำสุดที่ 31.97 โดยแนวโน้มดังกล่าวแสดงให้เห็นว่า Project ที่มี Test Cases จำนวนมากไม่ได้จำเป็นต้องมี Suite Length สูงที่สุดเสมอไป เนื่องจากความยาวของ Test แต่ละชุดแตกต่างกัน

5.2 Zest

ตารางที่ 8 ค่าเฉลี่ยคุณลักษณะของชุดกรณีทดสอบที่สร้างโดย Zest จำแนกตามโครงการ Defects4J

Project	JUnit Tests	Suite Length	Fuzz Time
Chart	50.81	61.15	120.12s
Cli	43.82	49.00	120.10s
Closure	27.25	28.65	119.93s
Codec	101.94	102.11	120.22s
Collections	53.57	56.82	125.93s
Compress	48.06	49.13	121.98s
Csv	16.75	19.25	126.88s
Gson	43.72	52.61	127.56s
JacksonCore	30.38	30.38	127.58s
JacksonDataBind	25.47	28.41	126.60s
JacksonXml	12.50	12.50	126.83s
Jsoup	75.29	225.65	120.00s
JXPath	45.41	193.34	122.14s
Lang	68.84	294.62	121.59s
Math	39.57	250.42	203.39s
Mockito	23.29	31.97	188.38s
Time	121.36	119.28	123.68s

จากตารางที่ 8 ผลการทดลองด้าน Test Suite Characteristics ของ Zest พิจารณาจากจำนวน JUnit Tests, Suite Length และ Fuzz Time พบว่าขนาดของ Test Suite และระยะเวลาในการ Fuzzing มีความแตกต่างกันระหว่าง Defects4J Project

ในด้าน จำนวน *JUnit Tests* พบว่า *Codec* มีจำนวน *JUnit Tests* เฉลี่ยสูงที่สุดที่ 101.94 Tests รองลงมาคือ *Jsoup* ที่ 75.29 Tests และ *Lang* ที่ 68.84 Tests ขณะที่ *JacksonXml* มีจำนวน *JUnit Tests* เฉลี่ยต่ำที่สุดที่ 12.50 Tests รองลงมาคือ *Csv* ที่ 16.75 Tests

สำหรับ *Suite Length* พบว่า *Jsoup* มีค่าเฉลี่ยสูงที่สุดที่ 225.65 รองลงมาคือ *JXPath* และ *Math* ที่ 193.34 และ 250.42 ตามลำดับ ขณะที่ *JacksonXml* มีค่า *Suite Length* ต่ำที่สุดที่ 12.50 ทั้งนี้ ค่า *Suite Length* ไม่ได้แปรผันตามจำนวน *JUnit Tests* โดยตรง เนื่องจาก *Test* แต่ละชุดอาจมีจำนวน *Statement* แตกต่าง

ในด้าน *Fuzz Time Zest* ใช้เวลาใกล้เคียงกันในหลาย *Project* โดยส่วนใหญ่อยู่ประมาณ 120–128 วินาที อย่างไรก็ตาม *Math* และ *Mockito* มี *Fuzz Time* สูงกว่า โดยมีค่า 203.39 วินาที และ 188.38 วินาที ตามลำดับ ขณะที่ *Jsoup* มีค่า *Fuzz Time* 120.00 วินาที ซึ่งสะท้อนว่าระยะเวลาในการ *Fuzzing* ขึ้นอยู่กับลักษณะของ *Project* และการทำงานของกระบวนการค้นหา *Test Case*

โดยรวม *Zest* สามารถสร้าง *Test Suite* ที่มีจำนวน *JUnit Tests* และ *Suite Length* แตกต่างกันตาม *Project* ขณะที่ *Fuzz Time* ของ *Project* ส่วนใหญ่มีค่าอยู่ในช่วงใกล้เคียงกัน และมีบาง *Project* ที่ใช้เวลามากกว่าช่วงดังกล่าว

5.3 Gemini

ตารางที่ 9 คุณลักษณะการสร้างกรณีทดสอบด้วย Gemini จำแนกตามโครงการ Defects4J

<i>Project</i>	<i>Bugs</i>	<i>Tokens</i>	<i>Refinement</i>
Chart	20	18353	0.43
Cli	39	18804	0.86
Closure	162	26191	0.83
Codec	18	16490	0.33
Collections	28	14537	0.29
Compress	46	20341	0.45
Csv	16	14416	0.29
Gson	18	15867	0.29
JacksonCore	25	29601	0.53
JacksonDataBind	48	24830	0.70
JacksonXml	6	16344	0.67

Jsoup	90	17778	0.47
JXPath	22	20982	0.85
Lang	60	22607	0.18
Math	93	16477	0.34
Mockito	38	14598	0.70
Time	25	25171	0.70

จากตารางที่ 9 ผลการทดลองด้าน *Test Suite Characteristics* ของ *Gemini-3.7-flash* พิจารณาจาก *Tokens Used* และ *Refinement Rounds* ของแต่ละ *Defects4J Project* พบว่าต้นทุนในการสร้างชุดทดสอบมีความแตกต่างกันตาม *Project* เช่นเดียวกับผลที่พบใน *GWO* และ *Zest*

ในด้าน *Tokens Used* พบว่า *JacksonCore* มีค่าเฉลี่ยสูงที่สุดที่ 29,601 *Token* รองลงมาคือ *Closure* ที่ 26,191 *Token* และ *JacksonDatabind* ที่ 24,830 *Token* ขณะที่ *Csv* มีค่าเฉลี่ยต่ำที่สุดที่ 14,416 *Token* รองลงมาคือ *Mockito* ที่ 14,598 *Token*

สำหรับ *Refinement Rounds* ซึ่งสะท้อนจำนวนรอบที่ต้องแก้ไขชุดทดสอบก่อนคอมไพล์ผ่าน พบว่า *Cli* มีค่าเฉลี่ยสูงที่สุดที่ 0.86 รอบ รองลงมาคือ *JXPath* ที่ 0.85 รอบ และ *Closure* ที่ 0.83 รอบ ขณะที่ *Lang* มีค่าต่ำที่สุดที่ 0.18 รอบ ทั้งนี้ *Tokens Used* ไม่ได้แปรผันตรงกับ *Refinement Rounds* เสมอไป เช่น *JacksonCore* ใช้ *Token* สูงสุดแต่มี *Refinement* เพียง 0.53 รอบ ขณะที่ *Cli* ใช้ *Token* ต่ำกว่าแต่กลับมี *Refinement* สูงสุดที่ 0.86 รอบ

เมื่อพิจารณาจำนวน *Bug* ร่วมกับ *Tokens Used* และ *Refinement Rounds* พบว่า *Math* มี *Bug* มากถึง 93 *Bug* แต่ใช้ *Token* เฉลี่ยเพียง 16,477 และ *Refinement* แค่ 0.34 รอบ ขณะที่ *Closure* มี *Bug* 162 *Bug* และใช้ทั้ง *Token* สูง (26,191) และ *Refinement* สูง (0.83 รอบ) สะท้อนว่า *Closure* เป็นโครงการที่มีความซับซ้อนทางโครงสร้างสูงกว่า *Math* แม้จะมีจำนวน *Bug* โกล้เคียงหรือมากกว่าก็ตาม

โดยรวม *Gemini-3.7-flash* มีรูปแบบการใช้ทรัพยากรที่ขึ้นอยู่กับความซับซ้อนของ *Source Code* ในแต่ละ *Project* มากกว่าจำนวน *Bug* เพียงอย่างเดียว ซึ่งสอดคล้องกับแนวโน้มที่พบใน *GWO* และ *Zest* เช่นกัน

5.4 Deepseek

ตารางที่ 10 คุณลักษณะการสร้างกรณีทดสอบด้วย DeepSeek จำแนกตามโครงการ Defects4J

<i>Project</i>	<i>Bugs</i>	<i>Tokens</i>	<i>Refinement</i>
Chart	26	34936	0.74
Cli	29	33795	0.97
Closure	25	37915	0.85
Codec	17	25902	0.52
Collections	27	18674	0.56
Compress	45	39548	0.64
Csv	16	19659	0.67
Gson	18	26897	0.78
JacksonCore	26	46510	0.91
JacksonDataBind	63	21013	0.85
JacksonXml	6	23357	1.00
Jsoup	93	36682	0.66
JXPath	22	40029	0.94
Lang	60	36790	0.61
Math	106	36023	0.65
Mockito	38	33176	0.63
Time	25	39820	0.84

จากตารางที่ 10 ผลการทดลองด้าน *Test Suite Characteristics* ของ *DeepSeek-v4-flash* พิจารณาจาก *Tokens Used* และ *Refinement Rounds* ของแต่ละ *Defects4J Project* พบว่าต้นทุนในการสร้างชุดทดสอบมีความแตกต่างกันตาม *Project* เช่นเดียวกับผลที่พบใน *GWO*, *Zest* และ *Gemini-3.7-flash* ในด้าน *Tokens Used* พบว่า *JacksonCore* มีค่าเฉลี่ยสูงที่สุดที่ 46,510 Token รองลงมาคือ *JXPath* ที่ 40,029 Token และ *Time* ที่ 39,820 Token ขณะที่ *Collections* มีค่าเฉลี่ยต่ำที่สุดที่ 18,674 Token รองลงมาคือ *Csv* ที่ 19,659 Token

สำหรับ *Refinement Rounds* พบว่า *JacksonXml* มีค่าเฉลี่ยสูงที่สุดที่ 1.00 รอบ รองลงมาคือ *Cli* ที่ 0.97 รอบ และ *JXPath* ที่ 0.94 รอบ ขณะที่ *Codec* มีค่าต่ำที่สุดที่ 0.52 รอบ รองลงมาคือ *Collections*

ที่ 0.56 รอบ ทั้งนี้ *Tokens Used* ไม่ได้แปรผันตรงกับ *Refinement Rounds* เสมอไป เช่น *JacksonDatabind* ใช้ *Token* ค่อนข้างต่ำ (21,013) แต่มี *Refinement* สูงถึง 0.85 รอบ เช่นเดียวกับ *JacksonXml* ที่ใช้ *Token* ปานกลาง (23,357) แต่กลับมี *Refinement* สูงสุดที่ 1.00 รอบ แสดงว่าชุดทดสอบของทั้งสองโครงการนี้ต้องถูกแก้ไขซ้ำบ่อยแม้ *Prompt* จะไม่ซับซ้อนมากนัก

เมื่อพิจารณาจำนวน *Bug* ร่วมกับ *Tokens Used* และ *Refinement Rounds* พบว่า *Math* มี *Bug* มากที่สุดถึง 106 *Bug* แต่ใช้ *Token* เฉลี่ยเพียง 36,023 และ *Refinement* 0.65 รอบ ซึ่งอยู่ในระดับปานกลาง ขณะที่ *JacksonCore* มี *Bug* เพียง 26 *Bug* แต่ใช้ทั้ง *Token* สูงสุด (46,510) และ *Refinement* สูง (0.91 รอบ) สะท้อนว่า *JacksonCore* เป็นโครงการที่มีความซับซ้อนทางโครงสร้างต่อ *Bug* สูงกว่า *Math* มาก แม้จะมีจำนวน *Bug* น้อยกว่า

โดยรวม *DeepSeek-v4-flash* มีรูปแบบการใช้ทรัพยากรที่ขึ้นอยู่กับความซับซ้อนของ *Source Code* ในแต่ละ *Project* มากกว่าจำนวน *Bug* เพียงอย่างเดียว เช่นเดียวกับแนวโน้มที่พบในทั้ง *GWO*, *Zest* และ *Gemini-3.7-flash* และเมื่อเทียบกับ *Gemini-3.7-flash* พบว่า *DeepSeek-v4-flash* ใช้ *Token* เฉลี่ยต่อ *Bug* สูงกว่าในแทบทุก *Project* (เช่น *JacksonCore* 46,510 เทียบกับ 29,601 ของ *Gemini*) ขณะที่ *Refinement Rounds* ของ *DeepSeek-v4-flash* ก็สูงกว่า *Gemini-3.7-flash* ในโปรเจกส่วนใหญ่เช่นกัน ซึ่งสอดคล้องกับผลด้าน *Line Coverage* ที่ *Gemini-3.7-flash* ทำได้ดีกว่าในภาพรวม

6. อภิปรายผล

จากผลการทดลองกับ *Defects4J* จำนวน 17 โครงการ รวม 854 *Bug* พบว่าจำนวน *Bug* ที่สามารถนำมาทดลองและได้ผลลัพธ์ที่สามารถนำมาวิเคราะห์ได้แตกต่างกันระหว่างแต่ละวิธี โดย *Zest* มีข้อมูลการทดลองมากที่สุด 809 *Bug* รองลงมาคือ *Gemini-3.7-flash* 447 *Bug*, *GWO* 373 *Bug* และ *DeepSeek-v4-flash* 273 *Bug* ความแตกต่างดังกล่าวสะท้อนถึงข้อจำกัดของแต่ละแนวทางในการสร้างและประมวลผล *Test Suite* เช่น การสร้าง *Test Case* ไม่สำเร็จ การ *Compile* ไม่ผ่าน หรือไม่สามารถเก็บข้อมูล *Coverage* ได้ ดังนั้น การเปรียบเทียบด้าน *Coverage* จึงควรพิจารณาบนชุด *Bug* ที่มีข้อมูลร่วมกัน เพื่อหลีกเลี่ยงผลกระทบจากจำนวนข้อมูลที่ไม่เท่ากัน

ในด้าน *Line Coverage* เมื่อพิจารณาเฉพาะ *Common Bugs* จำนวน 249 *Bug* ซึ่งทั้ง 4 วิธีมีข้อมูลที่สามารถเปรียบเทียบร่วมกันได้ พบว่า *Gemini-3.7-flash* มีค่า *Overall Line Coverage* 86.26% ขณะที่ *GWO*, *DeepSeek-v4-flash* และ *Zest* มีค่า 79.98%, 77.14% และ 41.92% ตามลำดับ ผลลัพธ์แสดงให้เห็นว่าความสามารถในการครอบคลุมบรรทัดของโปรแกรมมีความแตกต่างกันตามแนวทางการสร้าง *Test Suite* และลักษณะของแต่ละโครงการ อย่างไรก็ตาม ค่า *Coverage* ที่สูงไม่ได้หมายความว่าชุดทดสอบสามารถตรวจจับ *Bug* ได้มากกว่า เนื่องจาก *Coverage* เป็นตัวชี้วัดการเข้าถึงส่วนต่าง ๆ ของโปรแกรม และควรพิจารณาร่วมกับตัวชี้วัดด้านการตรวจจับ *Fault* และคุณภาพของ *Test Case*

เมื่อพิจารณาเป็นรายโครงการ ความแตกต่างของ Line Coverage มีความชัดเจนในบางโครงการ เช่น Closure ซึ่ง Gemini-3.7-flash มีค่า 89.35% ขณะที่ GWO มีค่า 5.23% และ Zest มีค่า 5.80% ในทางกลับกัน GWO มีค่า Coverage สูงในบางโครงการ เช่น Math ที่ 91.39% และ JacksonDatabind ที่ 94.87% ส่วน Csv เป็นโครงการที่ทั้ง GWO, Gemini และ DeepSeek มีค่า Coverage สูงกว่า 90% ข้อมูลดังกล่าวแสดงให้เห็นว่าไม่มีรูปแบบเดียวที่สามารถอธิบายผลลัพธ์ของทุกโครงการได้ เนื่องจากโครงสร้างของ Source Code, จำนวนและลักษณะของ Bug รวมถึง Class ที่เกี่ยวข้องกับ Bug มีผลต่อความสามารถของแต่ละวิธีในการสร้าง Test Case

สำหรับ Branch Coverage ผลการทดลองของ GWO และ Zest แสดงให้เห็นความแตกต่างระหว่างวิธีและระหว่างโครงการเช่นเดียวกับ Line Coverage โดย GWO มีค่า Branch Coverage ตั้งแต่ 25.17% ใน JacksonXml ถึง 83.85% ใน Cli ขณะที่ Zest มีค่าตั้งแต่ 3.75% ใน JacksonXml ถึง 62.48% ใน Codec ทั้งนี้ Codec และ Closure เป็นตัวอย่างที่ผลลัพธ์ของ Zest สูงกว่า GWO ตามข้อมูลในชุด Common Bugs ที่ใช้วิเคราะห์ แสดงให้เห็นว่า Coverage แต่ละประเภทอาจให้รูปแบบผลลัพธ์แตกต่างกัน และไม่ควรใช้ตัวชี้วัดเพียงประเภทเดียวในการอธิบายประสิทธิภาพของ Test Generation

ในด้าน Condition Coverage ซึ่งสามารถวิเคราะห์ร่วมกันระหว่าง Gemini-3.7-flash และ DeepSeek-v4-flash ได้จาก Common Bugs จำนวน 288 Bug พบว่า Gemini มี Overall Condition Coverage 60.03% และ DeepSeek มีค่า 48.64% โดยมีความแตกต่าง 11.39 จุดเปอร์เซ็นต์ เมื่อพิจารณารายโครงการ Gemini มีค่ามากกว่า DeepSeek ใน 14 จาก 17 โครงการ ขณะที่ DeepSeek มีค่ามากกว่าใน Gson, JacksonXml และ Mockito ผลดังกล่าวแสดงให้เห็นว่าความสามารถในการครอบคลุมเงื่อนไขมีความแตกต่างตามโครงการ และผลของ Line Coverage ไม่จำเป็นต้องสอดคล้องกับ Condition Coverage ในทุกกรณี

สำหรับ สาเหตุของความล้มเหลวในการสร้างชุดทดสอบด้วย AI พบรูปแบบข้อผิดพลาดที่เกี่ยวข้องกับการตีความโครงสร้างของ Source Code ได้แก่ การอ้างอิง Class หรือ Package ที่ไม่มีอยู่จริง การระบุจำนวน Argument ของ Constructor หรือ Method ไม่ตรงกับ Signature การคาดเดาชนิดข้อมูลที่ Method คืนค่าผิด และการ Override Method ใน Anonymous หรือ Inner Class ไม่ครบถ้วน ปัญหาเหล่านี้ส่งผลโดยตรงต่อความสามารถในการ Compile และรัน Test Suite โดยเฉพาะในโครงการที่มีโครงสร้าง Class และ Interface ที่ซับซ้อน ดังนั้น ความล้มเหลวของ AI ไม่ได้เกิดจากการสร้าง Test Case ที่ไม่ครอบคลุมเพียงอย่างเดียว แต่รวมถึงความถูกต้องทางโครงสร้างและความสอดคล้องกับ API ของ Source Code ด้วย

ในด้าน Test Suite Characteristics ของ GWO และ Zest พบว่าขนาดของ Test Suite แตกต่างกันอย่างมากระหว่างโครงการ สำหรับ GWO จำนวน Test Cases เฉลี่ยมีค่าตั้งแต่ 8.72 ใน Mockito ถึง 154.68 ใน Lang ขณะที่ Suite Length มีค่าตั้งแต่ 31.97 ใน Mockito ถึง 535.62 ใน JacksonCore ส่วน Zest มีจำนวน JUnit Tests เฉลี่ยตั้งแต่ 12.50 ใน JacksonXml ถึง 101.94 ใน Codec และ Suite Length ตั้งแต่ 12.50 ใน JacksonXml ถึง

294.62 ใน Lang ตามข้อมูลที่ใช้ในการทดลอง ความแตกต่างนี้แสดงให้เห็นว่าจำนวน Test Case และความยาวของ Test Suite ขึ้นอยู่กับโครงสร้างและลักษณะของแต่ละโครงการ ไม่ได้เปลี่ยนแปลงในทิศทางเดียวกันเสมอไป

สำหรับเวลาในการค้นหาหรือ Fuzzing พบว่า GWO มี Search Time แตกต่างกันตามโครงการ โดย Math มีค่า 203.39 วินาที ขณะที่หลายโครงการมีค่าใกล้เคียงประมาณ 121–122 วินาที ส่วน Zest มี Fuzz Time ส่วนใหญ่อยู่ใกล้ช่วง 120–128 วินาที แต่ Math และ Mockito มีค่า 203.39 และ 188.38 วินาทีตามลำดับ ผลดังกล่าวสะท้อนว่าเวลาในการค้นหา Test Case ไม่ได้ขึ้นอยู่กับจำนวน Bug เพียงอย่างเดียว แต่เกี่ยวข้องกับกระบวนการค้นหาและลักษณะของพื้นที่การค้นหาของแต่ละโครงการด้วย

สำหรับ Gemini-3.7-flash และ DeepSeek-v4-flash ไม่สามารถใช้ Test Cases และ Suite Length ในรูปแบบเดียวกับ GWO และ Zest ได้จากข้อมูลการทดลองที่จัดเก็บไว้ จึงพิจารณาจาก Tokens Used และ Refinement Rounds แทน โดย Gemini มีค่า Tokens Used เฉลี่ยต่อ Bug ต่ำกว่า DeepSeek ในโครงการส่วนใหญ่ ขณะที่ DeepSeek มีค่า Refinement Rounds สูงกว่า Gemini ในโครงการส่วนใหญ่ อย่างไรก็ตาม Tokens Used และ Refinement Rounds เป็นตัวชี้วัดลักษณะการทำงานของกระบวนการสร้าง Test Suite ไม่ใช่ตัวชี้วัดคุณภาพโดยตรง การใช้ Tokens มากกว่าจึงไม่สามารถสรุปได้ว่าชุดทดสอบมี Coverage หรือความสามารถในการตรวจจับ Fault สูงกว่าเสมอไป

เมื่อพิจารณาผลการทดลองโดยรวม พบว่าความแตกต่างระหว่างวิธีไม่ได้เกิดขึ้นในมิติเดียว แต่ประกอบด้วยความสามารถในการสร้าง Test Suite สำเร็จ, Line Coverage, Branch Coverage, Condition Coverage, ความถูกต้องในการ Compile และลักษณะของกระบวนการสร้าง Test Suite ผลการทดลองยังแสดงให้เห็นว่าแต่ละโครงการตอบสนองต่อวิธีการสร้าง Test แตกต่างกัน ดังนั้น การประเมิน Test Generation ควรพิจารณาหลายตัวชี้วัดร่วมกัน แทนการใช้ Coverage เพียงค่าเดียว

นอกจากนี้ การใช้ Common Bugs เป็นชุดข้อมูลสำหรับการเปรียบเทียบมีความสำคัญ เนื่องจากจำนวน Bug ที่แต่ละวิธีสามารถดำเนินการได้ไม่เท่ากัน การคำนวณ Coverage จาก Common Bugs ช่วยให้แต่ละวิธีถูกประเมินบนชุด Bug เดียวกันมากขึ้น ทำให้สามารถอธิบายความแตกต่างของ Coverage ระหว่างวิธีได้อย่างเหมาะสม ขณะเดียวกัน จำนวน Bug ที่ไม่สามารถนำมาวิเคราะห์ได้ยังเป็นข้อมูลสำคัญที่สะท้อนข้อจำกัดด้านการ Compile การสร้าง Test Suite และการประมวลผลของแต่ละวิธี และควรรายงานแยกจากผล Coverage แทนการละทิ้งข้อมูลดังกล่าว

โดยสรุป ผลการทดลองแสดงให้เห็นว่า ประสิทธิภาพของ Test Generation มีความสัมพันธ์กับทั้งวิธีการสร้าง Test และลักษณะของ Defects4J Project โดยผลลัพธ์ด้าน Coverage, ความสำเร็จในการสร้างชุดทดสอบ และต้นทุนของกระบวนการสร้าง Test มีความแตกต่างกันตามวิธีและโครงการ

บทที่ 5

สรุปผล

1. สรุปผลการทดสอบ

การศึกษานี้มีวัตถุประสงค์เพื่อทดลองและเปรียบเทียบประสิทธิภาพของวิธีการสร้างชุดทดสอบอัตโนมัติ 4 วิธี ได้แก่ Grey Wolf Optimizer (GWO), Zest, Gemini-3.7-flash และ DeepSeek-v4-flash บนโครงการ Defects4J จำนวน 17 โปรเจกต์ โดยพิจารณาจาก Line Coverage, Branch Coverage, Condition Coverage รวมถึงลักษณะของ Test Suite และข้อมูลการสร้างชุดทดสอบที่ได้จากแต่ละวิธี

จากผลการทดลองด้าน Line Coverage เมื่อพิจารณาเฉพาะ Common Bugs ที่สามารถเปรียบเทียบได้ทั้ง 4 วิธี จำนวน 249 Bugs พบว่า Gemini-3.7-flash มีค่า Line Coverage รวม 86.26% ขณะที่ GWO มีค่า 79.98%, DeepSeek-v4-flash มีค่า 77.14% และ Zest มีค่า 41.92% ผลดังกล่าวแสดงให้เห็นว่าความสามารถในการครอบคลุมโค้ดแตกต่างกันตามวิธีการสร้าง Test Suite และลักษณะของโครงการ โดยบางโครงการมีค่า Coverage แตกต่างกันอย่างชัดเจนระหว่างวิธีการ

สำหรับ Branch Coverage ผลการทดลองของ GWO และ Zest แสดงให้เห็นว่า GWO มีค่า Branch Coverage สูงกว่าในหลายโครงการ เช่น Chart, Cli, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, XPath, Lang และ Mockito ขณะที่ Zest มีค่าที่สูงกว่าในบางโครงการ เช่น Closure และ Codec ส่วน Math และ Time ไม่สามารถนำมาเปรียบเทียบได้ครบถ้วนเนื่องจากไม่มีข้อมูลที่ใช้งานได้

ด้าน Condition Coverage ซึ่งสามารถคำนวณเปรียบเทียบ Gemini-3.7-flash และ DeepSeek-v4-flash จาก Common Bugs จำนวน 288 Bugs พบว่า Gemini-3.7-flash มีค่า Condition Coverage รวม 60.03% และ DeepSeek-v4-flash มีค่า 48.64% โดย Gemini มีค่า Coverage สูงกว่าใน 14 จาก 17 โปรเจกต์ ขณะที่ DeepSeek มีค่าสูงกว่าใน Gson, JacksonXml และ Mockito

ในด้านลักษณะของ Test Suite พบว่า GWO สามารถสร้างชุดทดสอบที่มีจำนวน Test Cases และ Suite Length แตกต่างกันตามลักษณะของแต่ละโครงการ โดย Lang มีค่าเฉลี่ยจำนวน Test Cases 154.68 ซึ่งเป็นค่าที่สูงที่สุดในชุดข้อมูล GWO ขณะที่ JacksonCore มีค่าเฉลี่ย Suite Length 535.62 และ Math มีค่า Search Time เฉลี่ย 203.39 วินาที สำหรับ Zest ค่าเฉลี่ย JUnit Tests สูงสุดพบใน

Codec ที่ 101.94 Tests และค่า Suite Length สูงสุดพบใน Math ที่ 250.42 ขณะที่ระยะเวลา Fuzzing ของแต่ละโครงการส่วนใหญ่อยู่ในช่วงประมาณ 120–128 วินาที

สำหรับวิธีที่ใช้ Large Language Model พบว่า Gemini-3.7-flash และ DeepSeek-v4-flash มีปริมาณการใช้ Tokens และจำนวน Refinement Rounds แตกต่างกันตามโครงการ Gemini มีค่า Tokens สูงสุดใน JacksonCore ที่ 29,601 Tokens ส่วน DeepSeek มีค่าสูงสุดใน JacksonCore ที่ 46,510 Tokens ข้อมูลดังกล่าวสะท้อนให้เห็นว่าความซับซ้อนของโครงการและลักษณะของ Source Code มีส่วนเกี่ยวข้องกับปริมาณการประมวลผลที่ใช้ในการสร้างและปรับปรุง Test Suite

นอกจากนี้ยังพบปัญหาการสร้าง Test Case ของวิธีที่ใช้ AI เช่น การอ้างอิง Class หรือ Package ที่ไม่มีอยู่จริง การเรียก Method ด้วยจำนวน Parameter ไม่ตรงกับ Source Code การคาดเดา Return Type ไม่ถูกต้อง และการสร้าง Anonymous หรือ Inner Class ที่ Override Method ไม่ครบ ปัญหาเหล่านี้ส่งผลโดยตรงต่อความสามารถในการ Compile และการนำ Test Case ไปใช้วัด Coverage

โดยสรุป ผลการทดลองแสดงให้เห็นว่าวิธีการสร้าง Test อัตโนมัติแต่ละวิธีมีลักษณะและผลลัพธ์แตกต่างกัน ทั้งในด้าน Coverage, โครงสร้างของ Test Suite, เวลาในการค้นหา และปริมาณทรัพยากรที่ใช้

ตารางที่ 11 ภาพรวมค่าเฉลี่ยรวมทุก Project

Method	Avg Line Coverage	Avg Branch/Condition Coverage	Avg Cost
Gemini-3.7-flash	73.45%	64.82%	19,611.11 token
GWO	67.79%	63.82%	143.55 sec
DeepSeek-v4-flash	61.00%	53.95%	32,528.85 token
Zest	34.43%	25.66%	122.92 sec

จากตารางที่ 11 แสดงผลการเปรียบเทียบค่าเฉลี่ยความครอบคลุมของโค้ดและต้นทุนในการสร้างกรณีทดสอบของทั้ง 4 วิธี พบว่า Gemini-3.7-flash ให้ผลดีที่สุดทั้งสองตัวชี้วัด โดยมีค่าเฉลี่ย Line Coverage เท่ากับ 73.45% และ Branch/Condition Coverage เท่ากับ 64.82% รองลงมาคือ GWO ที่มีค่าเฉลี่ย Line Coverage 67.79% และ Branch/Condition Coverage 63.82% ถัดมาคือ DeepSeek-v4-flash ที่มีค่าเฉลี่ย 61.00% และ 53.95% ตามลำดับ ส่วน Zest ให้ผลต่ำที่สุด โดยมีค่าเฉลี่ย Line Coverage เพียง 34.43% และ Branch/Condition Coverage 25.66%

เมื่อพิจารณาในรายละเอียด Gemini-3.7-flash มี Line Coverage สูงกว่า GWO 5.66 จุดเปอร์เซ็นต์ แต่ Branch/Condition Coverage สูงกว่าเพียง 1.00 จุดเปอร์เซ็นต์ แสดงให้เห็นว่า วิธีแบบ search-based อย่าง GWO สามารถทดสอบเส้นทางการตัดสินใจได้ใกล้เคียงกับโมเดล ภาษาขนาดใหญ่ นอกจากนี้ GWO ยังมีส่วนต่างระหว่าง Line Coverage กับ Branch/Condition Coverage น้อยที่สุด (3.97 จุดเปอร์เซ็นต์) เมื่อเทียบกับ Gemini-3.7-flash (8.63), DeepSeek-v4-flash (7.05) และ Zest (8.77) ซึ่งสะท้อนว่า GWO สร้างกรณีทดสอบที่ครอบคลุมทั้งบรรทัดคำสั่ง และกิ่งการตัดสินใจได้สมดุลกว่าวิธีอื่น

ในด้านต้นทุน เมื่อเปรียบเทียบระหว่างโมเดลปัญญาประดิษฐ์ด้วยกัน DeepSeek-v4-flash ใช้โทเคนเฉลี่ย 32,528.85 โทเคน ซึ่งมากกว่า Gemini-3.7-flash ที่ใช้ 19,611.11 โทเคน ประมาณ 1.66 เท่า แต่กลับให้ความครอบคลุมต่ำกว่าทั้งสองตัวชี้วัด แสดงว่า Gemini-3.7-flash มีประสิทธิภาพเชิงต้นทุนดีกว่า ส่วนวิธีแบบดั้งเดิม GWO ใช้เวลาเฉลี่ย 143.55 วินาที มากกว่า Zest ที่ใช้ 122.92 วินาที เพียง 20.63 วินาที (ประมาณ 16.8%) แต่ให้ Line Coverage สูงกว่าเกือบ 2 เท่า และ Branch/Condition Coverage สูงกว่าประมาณ 2.5 เท่า ทั้งนี้ ต้นทุนของโมเดล ปัญญาประดิษฐ์วัดเป็นจำนวนโทเคน ขณะที่วิธีแบบดั้งเดิมวัดเป็นเวลา จึงไม่สามารถเปรียบเทียบ ต้นทุนข้ามกลุ่มได้โดยตรง

ผลที่ Zest ได้รับความครอบคลุมต่ำอาจเป็นเพราะ Zest เป็นเทคนิค fuzzing ที่ขึ้นกับ generator สำหรับสร้างข้อมูลนำเข้า หาก generator ไม่สามารถสร้างข้อมูลที่มีโครงสร้างหรือค่าที่ ตรงกับเงื่อนไขของโปรแกรมได้ กรณีทดสอบจึงเข้าถึงโค้ดได้เพียงบางส่วน

โดยสรุป Gemini-3.7-flash เป็นวิธีที่ให้ความครอบคลุมสูงสุดและใช้โทเคนน้อยกว่า DeepSeek-v4-flash ขณะที่ GWO เป็นทางเลือกที่ให้ Branch/Condition Coverage ใกล้เคียงกับ Gemini-3.7-flash โดยไม่ต้องใช้โมเดลภาษาขนาดใหญ่ ส่วน Zest มีความเหมาะสมน้อยที่สุด สำหรับการสร้างกรณีทดสอบในบริบทของการทดลองนี้

2. ปัญหาอุปสรรค และแนวทางการแก้ไข

2.1 ปัญหาที่ 1

ความแตกต่างของโครงสร้างและขนาดของโครงการ

โครงการใน Defects4J มีจำนวน Bugs ขนาด Source Code และโครงสร้าง Class ที่แตกต่างกัน ส่งผลให้ บางวิธีสามารถสร้าง Test Case ได้ในบางโครงการ แต่ไม่สามารถดำเนินการได้ครบทุก Bug

แนวทางการแก้ไข: กำหนดรูปแบบการทดลองและเกณฑ์การเก็บข้อมูลให้เหมือนกันทุกโครงการ และใช้ Common Bugs เป็นส่วนหนึ่งของการเปรียบเทียบ เพื่อให้แต่ละวิธีถูกเปรียบเทียบภายใต้ชุด Bugs เดียวกัน

2.2 ปัญหาที่ 2

Compilation ของ Test Case ที่สร้างโดย AI

Test Case ที่สร้างโดย Gemini และ DeepSeek บางรายการไม่สามารถ Compile ได้ เนื่องจากการอ้างอิง Class หรือ Package ที่ไม่มีอยู่จริง การส่ง Parameter ไม่ตรงกับ Method และการขาด Return Type ผิด

แนวทางการแก้ไข: ตรวจสอบ Compilation ก่อนนำ Test Case ไปวัด Coverage และบันทึกสาเหตุของ Compilation Failure แยกตามประเภท เพื่อป้องกันไม่ให้ Test Case ที่ไม่สามารถใช้งานได้ถูกนำมาคำนวณ Coverage โดยไม่เหมาะสม

2.3 ปัญหาที่ 3

ความแตกต่างของรูปแบบผลลัพธ์จากแต่ละเครื่องมือ

GWO, Zest, Gemini และ DeepSeek มีรูปแบบ Log และข้อมูลผลลัพธ์ที่แตกต่างกัน ทำให้การรวบรวมข้อมูลจำเป็นต้องมีขั้นตอน Parsing ที่แตกต่างกัน

แนวทางการแก้ไข: พัฒนาสคริปต์สำหรับอ่านข้อมูลจากแต่ละแหล่งโดยอัตโนมัติ และกำหนดรูปแบบข้อมูลกลาง เช่น Project, Bug ID, Coverage และผลการ Compile ก่อนนำข้อมูลมาวิเคราะห์ร่วมกัน

2.4 ปัญหาที่ 4

จำนวน Bugs ที่สามารถนำมาเปรียบเทียบได้ไม่เท่ากัน

ในการทดลองจริงแต่ละวิธีไม่สามารถสร้างผลลัพธ์ที่ใช้งานได้ครบทุก Bug ดังนั้นการนำค่าเฉลี่ยจาก Bugs ทั้งหมดของแต่ละวิธีมาเปรียบเทียบโดยตรงอาจทำให้เกิดความแตกต่างด้านจำนวนข้อมูล

แนวทางการแก้ไข: แยกการวิเคราะห์ออกเป็น 2 ส่วน ได้แก่ ผลลัพธ์จาก Bugs ทั้งหมดที่มีข้อมูลของแต่ละวิธี และผลลัพธ์จาก Common Bugs ซึ่งใช้ Bug ID เดียวกันในการเปรียบเทียบระหว่างวิธี

2.5 ปัญหาที่ 5

ข้อจำกัดด้านเวลาและทรัพยากรในการทดลอง

การสร้าง Test Suite โดยเฉพาะกระบวนการ Search-based Testing และการสร้าง Test Case ด้วย AI ต้องใช้เวลาและทรัพยากรในการประมวลผลแตกต่างกันในแต่ละโครงการ

แนวทางการแก้ไข: กำหนด Search Budget และรูปแบบการทดลองให้เป็นมาตรฐาน รวมถึงเก็บข้อมูล Search Time, Tokens และ Refinement Rounds เพื่อใช้ประกอบการวิเคราะห์ต้นทุนของแต่ละวิธี

2.6 ปัญหาที่ 6

ปัญหาความไม่สมบูรณ์ของข้อมูลผลการทดลอง

บางโครงการหรือบางวิธีไม่มีข้อมูล Coverage บางประเภท ทำให้ไม่สามารถคำนวณตัวชี้วัดบางรายการได้

แนวทางการแก้ไข: ไม่แทนค่าข้อมูลที่ไม่มีด้วยศูนย์ แต่กำหนดเป็น N/A และคำนวณค่าเฉพาะข้อมูลที่มีหลักฐานเพียงพอ เพื่อหลีกเลี่ยงการทำให้ค่าเฉลี่ยคลาดเคลื่อน

3. ข้อเสนอแนะ

จากผลการทดลองและปัญหาที่พบ สามารถเสนอแนะแนวทางสำหรับการพัฒนาการทดลองในอนาคตได้ดังนี้

3.1 เพิ่มจำนวนรอบการทดลอง

การทดลองในอนาคตควรดำเนินการหลายรอบต่อ Bug และต่อวิธีการ เพื่อศึกษาความแปรปรวนของผลลัพธ์และลดผลกระทบจากความแตกต่างที่เกิดจาก Randomness ของ Search-based Testing และการสร้าง Test Case ด้วย AI

3.2 เพิ่มตัวชี้วัดในการประเมินผล

นอกจาก Line Coverage, Branch Coverage และ Condition Coverage ควรพิจารณาตัวชี้วัดอื่น เช่น Mutation Score, Fault Detection Rate, Test Execution Time, Test Suite Size และ Compilation Success Rate เพื่อให้สามารถวิเคราะห์คุณภาพของ Test Suite ได้ครอบคลุมมากขึ้น

3.3 ปรับปรุงกระบวนการสร้าง Test Case ด้วย AI

ควรเพิ่มขั้นตอนตรวจสอบ Source Code และ API ก่อนสร้าง Test Case เพื่อช่วยลดปัญหาการอ้างอิง Class, Method หรือ Parameter ที่ไม่ถูกต้อง รวมถึงเพิ่มขั้นตอน Compile และ Feedback Loop เพื่อให้ AI สามารถแก้ไข Test Case ที่ไม่สามารถ Compile ได้โดยอัตโนมัติ

3.4 พัฒนาเกณฑ์การเปรียบเทียบที่เป็นมาตรฐาน

การทดลองในอนาคตควรกำหนด Common Bugs และเกณฑ์การวัดผลให้ชัดเจนตั้งแต่เริ่มต้น รวมถึงกำหนดวิธีการจัดการกับข้อมูลที่ขาดหายหรือไม่สามารถ Compile ได้ เพื่อให้ผลการทดลองของแต่ละวิธีสามารถเปรียบเทียบกันได้อย่างเป็นระบบ

3.5 วิเคราะห์ความสัมพันธ์ระหว่างลักษณะโครงการกับผลการทดสอบ

ควรศึกษาปัจจัยของ Source Code เช่น จำนวน Class, จำนวน Method, จำนวน Branch และความซับซ้อนของโค้ด เพื่อวิเคราะห์ว่าปัจจัยใดมีความสัมพันธ์กับ Coverage, จำนวน Test Case, Search Time, Tokens และจำนวน Refinement Rounds

3.6 ขยายการทดลองไปยังเครื่องมือและโมเดลอื่น

สามารถเพิ่ม Search-based Testing Algorithm หรือ AI Model อื่นเข้ามาในการทดลอง เพื่อศึกษาความแตกต่างของแนวทางการสร้าง Test Case และเพิ่มความครอบคลุมของการประเมินผล

3.7 จัดทำระบบวิเคราะห์ผลแบบอัตโนมัติ

ควรพัฒนากระบวนการตั้งแต่การรันการทดลอง การเก็บ Log การตรวจสอบ Compilation การคำนวณ Coverage และการสร้างกราฟให้อยู่ใน Pipeline เดียวกัน เพื่อลดข้อผิดพลาดจากการประมวลผลข้อมูลด้วยตนเองและทำให้สามารถทำซ้ำการทดลองได้ง่ายขึ้น

อ้างอิง

- Padhye, R., Lemieux, C., Sen, K., Papadakis, M., & Le Traon, Y. (2019). Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2019)*, 329–340.
- Padhye, R., Lemieux, C., & Sen, K. (2019). JQF: Coverage-Guided Property-Based Testing in Java. In *Proceedings of ISSTA 2019*.
- Efficient Test Case Reduction Using Grey Wolf Optimization Techniques. (2025). *EAI Endorsed Transactions*.
- Kumari, S., Jindal, S., & Sharma, A. (2025). Test case optimization using grey wolf algorithm. *Software Quality Journal*, 33, 20.
- Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., & Yin, J. (2024). ChatUniTest: A framework for LLM-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (FSE Companion '24). ACM.
- Kang, S., Yoon, J., & Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE.
- Ouedraogo, W. C., Kabore, A. K., Li, Y., Tian, H., Koyuncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2026). Prompt engineering in LLMs for automated unit test generation: A large-scale study. *Empirical Software Engineering*, 31(4), 1–58.
- Plein, L., Ouédraogo, W. C., Klein, J., & Bissyandé, T. F. (2024). Automatic generation of test cases based on bug reports: A feasibility study with large language models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (ICSE-Companion '24). ACM.
- Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2023). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1), 85–105.
- Wang, G., Xu, Q., Briand, L. C., Guo, Z., & Liu, K. (2026). *Call-chain-aware LLM-based test generation for Java projects*. arXiv.